硕士学位论文

（学术学位论文）

基于移动端 CPU-NPU 的深度学习模型部署与优化技术研究

RESEARCH ON DEPLOYMENT AND OPTIMIZATION TECHNOLOGY FOR DEEP LEARNING MODELS BASED ON MOBILE CPU-NPU

赵俊

哈尔滨工业大学

2025 年 3 月

中国知网 https://www.cnki.net

国内图书分类号：TP302.1　　学校代码：10213
国际图书分类号：004.45　　密级：公开

硕士学位论文

基于移动端 CPU-NPU 的深度学习模型部署与优化技术研究

硕 士 研 究 生：赵俊
导　　师：张展教授
申 请 学 位：工学硕士
学　　科：计算机科学与技术
所 在 单 位：计算学部
答 辩 日 期：2025 年 3 月
授予学位单位：哈尔滨工业大学

中国知网　https://www.cnki.net

Classified Index: TP302.1
U.D.C:004.45

Dissertation for the Master Degree

RESEARCH ON DEPLOYMENT AND OPTIMIZATION TECHNOLOGY FOR DEEP LEARNING MODELS BASED ON MOBILE CPU-NPU

Candidate： Zhao Jun
Supervisor： Prof.Zhang Zhan
Academic Degree Applied for： Master of Engineering
Speciality： Computer Science and Technology
Affiliation： Faculty of Computing
Date of Defence： March，2025
Degree-Conferring-Institution： Harbin Institute of Technology

中国知网 https://www.cnki.net

摘　要

随着移动设备和深度学习技术的迅猛发展，特别是大模型应用日趋成熟，智能应用在移动平台上需求激增，因此在移动平台上亟需高效的深度学习模型部署技术。然而，当前移动设备上的模型部署面临计算资源有限与计算资源分配不合理两大挑战，使得深度学习任务无法被高效地部署。本文聚焦于高性能片上系统（SoC）RK3588，特别是其内置的高效神经网络处理器单元（NPU），旨在通过一系列优化措施提高模型推理速度并缩短任务响应时间。

具体而言，本文首先全面分析了 RK3588 异构融合处理器的特点及其现有问题，并深入探讨了两大主要模型后端推理框架即 ONNX Runtime 与 RKNN Runtime 在不同计算规模下的算子性能差异。针对 NPU 推理框架对高维算子支持不足的问题，本文提出了一种创新性的算子降维算法，确保优化后的计算图能够充分利用 CPU 和 NPU 的计算能力。

为解决 RKNN 推理框架中计算资源分配不可控的问题，本文开发了一个基于时延预测的数据传输预测模块，指导模型分区决策，并引入了零拷贝技术和内存复用策略，有效降低了模型的数据传输时间。同时，为了解决维度交换算子在 RKNN 模型转换过程中自动添加导致的性能瓶颈，本文设计了一种基于冗余算子组合的纯净矩阵乘法算子转换方法，实现了维度交换算子与矩阵乘法算子的完全分离，提高了整体效率并简化了模型结构。本文还提出了基于 EDF 的多模型调度策略，进一步提升了多任务场景下的模型推理速度和任务响应能力。此外，本文还构建了一个综合调控软件平台，实现了直观的界面用于模型优化配置与计算任务调度控制。

最终实验结果表明，使用本文提出的模型优化技术和调度策略可以减少多任务场景下约 65%的平均响应时间，显著提升了深度学习任务在 RK3588 上的执行效率。

关键词：计算图优化；模型分区；任务调度；深度学习模型部署；RK3588

中国知网　https://www.cnki.net

Abstract

With the rapid development of mobile devices and deep learning technologies, especially as large-scale models mature, there is a surge in demand for intelligent applications on mobile platforms. Therefore, efficient deployment technology for deep learning models on mobile platforms has become critical. However, current model deployments on mobile devices face two major challenges: limited computational resources and inefficient allocation of these resources, which impede the efficient deployment of deep learning tasks. This paper focuses on the high-performance system-on-chip (SoC) RK3588, particularly its built-in efficient neural network processing unit (NPU), aiming to improve model inference speed and reduce task response time through a series of optimization measures.

Specifically, this paper first comprehensively analyzes the characteristics of the RK3588 heterogeneous integrated processor and its existing issues, and delves into the performance differences of operators under different computational scales between the two main model backend inference frameworks, ONNX Runtime and RKNN Runtime. In response to the insufficient support for high-dimensional operators within the NPU inference framework, we propose an innovative operator dimensionality reduction algorithm that ensures the optimized computation graph can fully utilize the computational capabilities of both the CPU and NPU.

To address the issue of uncontrollable computational resource allocation within the RKNN inference framework, we have developed a data transmission prediction module based on latency prediction, guiding model partitioning decisions, and introduced zero-copy technology and memory reuse strategies, effectively reducing the data transmission time of models. Additionally, to tackle the performance bottleneck caused by automatically added dimension exchange operators during RKNN model conversion, we designed a pure matrix multiplication operator

中国知网 https://www.cnki.net

conversion method based on redundant operator combinations, achieving complete separation of dimension exchange operators from matrix multiplication operators, thereby improving overall efficiency and simplifying model structure. We also proposed a multi-model scheduling strategy based on Earliest Deadline First (EDF), further enhancing model inference speed and task response capability in multi-task scenarios. Furthermore, we constructed a comprehensive regulation software platform with an intuitive interface for model optimization configuration and computational task scheduling control.

The final experimental results show that using the model optimization techniques and scheduling strategies proposed in this paper can reduce the average response time by approximately 65% in multi-task scenarios, significantly improving the execution efficiency of deep learning tasks on RK3588.

Keywords： computation graph optimization, model partitioning, deep learning model deployment, task scheduling, rk3588

中国知网 https://www.cnki.net

目 录

第 1 章 绪论.....1
1.1 课题来源及研究的背景和意义.....1
1.1.1 课题的来源.....1
1.1.2 研究背景及意义.....1
1.2 国内外研究现状.....3
1.2.1 面向移动端的深度学习模型轻量化研究现状.....3
1.2.2 移动端神经网络处理器发展现状.....5
1.2.3 异构融合处理协同计算技术研究现状.....6
1.2.4 现有研究现状的不足.....8
1.3 本文的主要研究内容.....9
1.4 论文组织结构.....10
第 2 章 面向深度学习的 RK3588 处理器性能分析.....11
2.1 引言.....11
2.2 RK3588 处理器架构概述.....11
2.2.1 CPU 架构.....11
2.2.2 RKNPU 架构.....12
2.3 深度学习模型推理框架选择与分析.....14
2.3.1 面向 CPU 的推理框架 ONNX Runtime.....14
2.3.2 面向 RKNPU 的推理框架 RKNN Runtime.....16
2.4 算子性能测试与分析.....18
2.4.1 CPU 与 NPU 推理时间对比.....19
2.4.2 CPU 与 NPU 数据传输时间对比.....23
2.5 面向 RK3588 的深度学习推理优化方案.....23
2.6 本章小结.....24
第 3 章 面向 RKNPU 的高维算子降维算法研究.....25
3.1 引言.....25
3.2 模型高维算子耗时分析.....25
3.3 RKNN 算子定义.....27
3.4 基于维度合并的高维算子降维算法.....28
3.4.1 基于字典索引搜索的 Add 算子降维算法.....29
3.4.2 基于递增子序列合并的 Transpose 算子降维算法.....31
3.5 算法性能测试与实验结果分析.....33
3.6 本章小结.....35
第 4 章 基于时延预测的模型分区与调度算法研究.....36

中国知网 https://www.cnki.net

4.1 引言.....36
4.2 面向单一模型的基于时延预测的模型分区算法.....36
4.2.1 RKNN 零拷贝与内存复用结合.....37
4.2.2 基于冗余算子组合的纯净矩阵乘法算子转换算法.....40
4.2.3 基于支持向量回归的数据传输时延预测模块.....44
4.2.4 基于时延预测的模型分区算法.....46
4.3 面向多模型的最早截止时间优先调度策略.....48
4.3.1 Linux 系统调度策略.....48
4.3.2 基于 EDF 的多模型调度策略.....49
4.4 算法性能测试与实验结果分析.....50
4.4.1 模型分区实验与结果分析.....50
4.4.2 多模型调度实验结果分析.....51
4.5 本章小结.....52
第 5 章 实验平台设计与实现.....54
5.1 引言.....54
5.2 实验平台搭建.....54
5.2.1 平台整体框架.....54
5.2.2 应用软件设计与实现.....55
5.3 混合调度实验结果与分析.....57
5.3.1 实验设计.....57
5.3.2 模型选择.....58
5.3.3 实验指标.....59
5.3.4 实验结果与分析.....59
5.4 本章小结.....62
结论.....63
参考文献.....65

中国知网 https://www.cnki.net

第1章 绪 论

1.1 课题来源及研究的背景和意义

1.1.1 课题的来源

本课题来源于部委预研项目“面向应用的可穿戴计算体系研究”，该项目主要研究以新型可穿戴智能头盔为核心的移动端异构融合算力平台，在特定场景下执行多种深度神经网络模型的推理，例如目标检测、图像融合、图像分割、图像分裂。如何利用移动端有限的计算能力、最大化利用移动端异构处理器资源、提高模型的推理速度、提高计算任务的平均响应时间，是本课题主要的研究目的。

1.1.2 研究背景及意义

（1）研究背景

随着移动设备和深度学习技术的迅猛发展，移动端智能应用的需求日益增长。现代智能手机、平板电脑和其他便携式设备不仅需要处理传统的计算任务，还需要支持复杂的机器学习模型进行实时推理，以提供诸如图像识别、语音助手、个性化推荐等高级功能。然而，移动设备受限于其有限的计算资源（如处理器性能、内存容量和电池寿命），这对高效部署深度学习模型提出了重大挑战，特别是随着大模型的提出和广泛应用使得这一需求变得更加迫切。

为了解决这类问题，许多公司如华为、Qualcomm 和瑞芯微等为移动设备开发了专用的神经网络处理单元 (Neural network Processing Unit，NPU)。在 HUAWEI mate 10 pro 手机中搭载的 NPU 上运行 ResNet50[1]。（Residual Network-50）比在 CPU 上运行快 20 倍。Qualcomm Snapdragon 拥有专用于 AI 应用程序的的 NPU 芯片组。瑞芯微的 RK3588 芯片组集成了强大的 NPU，主要面向 ARM 架构的个人计算机（Personal Computer，PC）、边缘计算、个人移动互联网设备以及其他多媒体产品应用，在处理以卷积算子为主的深度学习模型如 YOLO[2]（You Only Look Once）系列时，相比传统 CPU，其推理速度提升了 10 倍以上，显著优化了移动端和嵌入式设备的 AI 性能。

在此背景下，可穿戴智能头盔被提出，相较于传统的桌面系统或服务器集群，本课题研究的智能头盔作为移动端设备具有便携性和移动性优势，使得计算资源可以随时随地被利用，尤其是在不具备固定设施的野外作业场景中。如图 1-1，本

中国知网 https://www.cnki.net

课题使用的智能头盔搭载了 RK3588 处理器，配备多种摄像头模组，如鱼眼深度摄像模组、红外模组、微光模组、双目显示模组等，能够在不同条件下获取丰富的图像数据，执行复杂的计算机视觉类深度学习任务如目标检测、图像融合、图像分割、图像分类等。

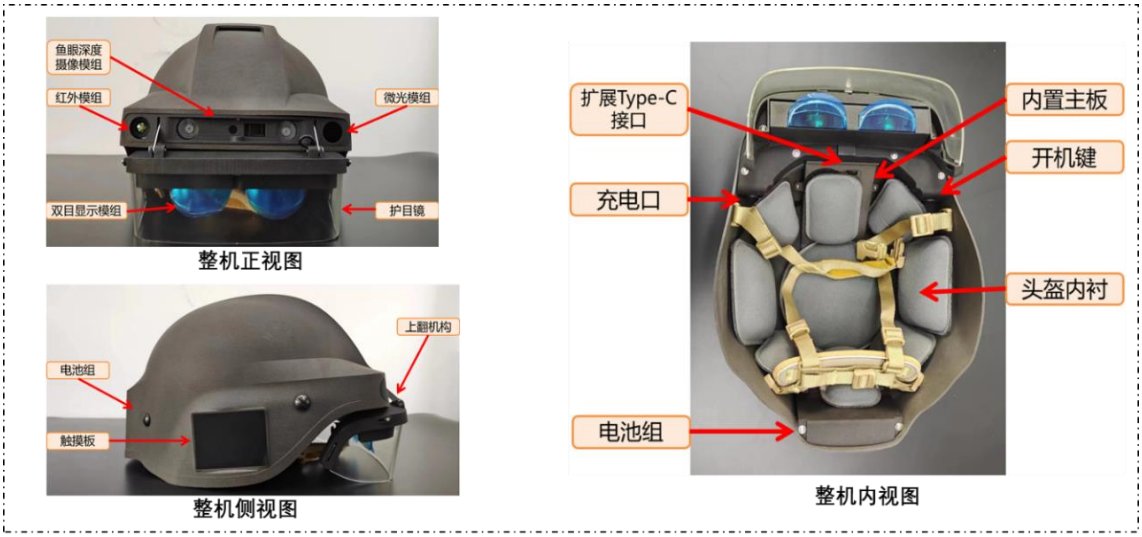


图 1-1 搭载 RK3588 处理器的智能头盔

（2）研究意义

尽管如此，现有的深度学习框架和工具链在利用这些新型硬件特性方面仍存在不足，特别是在算子支持程度、资源分配和任务调度等方面。

首先，目前大多数解决方案都专注于在单个人工智能加速器（如 GPU 或 NPU）上加速深度神经网络任务（Deep Neural Network，DNN）推理，但这些技术不能直接扩展以充分利用 CPU-NPU 异构体系结构。

其次，当前嵌入式设备搭载的异构处理器推理速度各有异同，例如 NPU 计算卷积与矩阵乘法速度远比 CPU 快，但 CPU 在数据搬运类算子上的表现以碾压的方式胜过 NPU，这将导致由一些特殊的模型结构引起的处理器利用率低下且推理速度慢的问题，另外当前 NPU 推理框架对高维算子的支持不足，计算资源分配不可控，对个性化开发的支持较低，特别像来自于瑞芯微 NPU 或 Google TPU 这样的硬件只有厂商提供的闭源推理框架可以使用，这些问题限制了模型性能的进一步提升。

最后，不同的计算资源之间的数据传输时间开销也成为提高异构处理器资源使用率的阻碍。因此，如何利用好移动端上的异构处理器资源中不同特点的处理器（即 CPU、NPU）来提高 DNN 模型推理速度是一项重大的挑战，也是是本课题的研究方向。

中国知网 https://www.cnki.net

1.2 国内外研究现状

深度神经网络（Deep neural networks，DNN）已成功应用于计算机视觉、自然语言处理、医疗保健等的各种问题，一些先进的模型在某些特定的数据集中甚至可以胜过人类[3,4]。许多基于 DNN 模型的应用程序已开发并在移动设备上运行。例如，基于 DNN 的应用程序可以将用户的语音识别为文本输入[5,6]，帮助旅行者识别建筑物和地标[7,8]，通过分析用户的声音来检测帕金森的疾病[9]。并且，伴随人工智能技术的加速演进，AI 大模型已成为全球科技竞争的新高地、未来产业的新赛道、经济发展的新引擎，发展潜力大、应用前景广。尽管当前的大模型已经取得了令人瞩目的成就，但在将其应用于更加广泛的场景特别是移动平台上时，仍然需要克服诸多技术障碍，因为这些模型虽然具有良好的精度，但它们也存在计算开销高的问题，即在移动设备上运行时延迟长。针对这些问题，各大厂商与学者开展了不尽相同的优化方面的研究。在模型优化层面主要使用模型压缩技术实现模型轻量化，在硬件方面主要表现为移动端神经网络处理器的高速发展，而随着移动端异构处理器的发展背景下，越来越多的有关模型部署的优化技术异构处理器的协同计算优化技术也被提出。

1.2.1 面向移动端的深度学习模型轻量化研究现状

目前，面向移动端的深度学习模型轻量化研究主要已取得了显著进展，主要分为轻量化模型结构设计、模型压缩、计算图替换。

（1）轻量化网络设计

深度可分离卷积的应用：许多轻量化网络如 Google 的 MobileNet 系列、旷视的 ShuffleNet 系列，都采用了深度可分离卷积来代替传统卷积。以 MobileNetV1 为例，其将标准卷积分解为深度卷积和逐点卷积，大大减少了计算量和参数量，同时在一定程度上保持了模型的性能，使得模型更适合移动端的部署[10]。一些研究尝试在轻量化网络中加入注意力机制，如 Hu J 等学者通过对通道特征的重标定，提升了模型的特征表示能力，且增加的计算量相对较小[11]。Li Y 学者重新审视了 Transformer 中的设计选择，并引入细粒度联合搜索策略，实现了低延迟和高参数效率，在 ImageNet-1k 上取得了较好的性能[12]。除了这些常见的方法，还有一些研究提出了新的网络结构。例如，Han K 等人通过生成 Ghost 特征图来减少计算量，同时保持较高的精度[13]；Li Y 和 Chen Y 等人则采用了微分解卷积和动态激活函数等技术，进一步提高了模型的效率[14]。

（2）模型压缩

中国知网 https://www.cnki.net

模型权重剪枝：剪枝技术包括非结构化剪枝和结构化剪枝。非结构化剪枝如 Han S 等人通过剪枝、量化和哈夫曼编码相结合的方式对深度神经网络进行压缩，其中量化部分展示了如何在不损失过多精度的前提下降低模型存储量，启发了后续研究人员在多技术融合压缩模型方面的深入探索，如结合不同的量化方案与结构化剪枝技术来平衡模型压缩比和准确性[15]。Tanaka H 等学者提出了一种无需数据的神经网络剪枝方法，通过迭代保护突触流来选择要保留的权重，这种方法可以在不牺牲精度的情况下显著减少模型大小[16]。Wen W 等人提出了学习结构化稀疏性的框架，通过正则化技术直接作用于卷积核或滤波器级别，从而实现对整个通道或层的删除[17]。

模型量化：量化方法一般是将模型的参数和激活值从高精度的数据类型（如 FP32）转换为低精度的数据类型（如 INT8），从而加快网络推理速度。为了减少量化带来的精度损失，研究人员提出了各种量化策略，如对称量化、非对称量化、数值对齐等。Jacob B 等人提出了一种将神经网络权重和激活函数量化为 8 位整数的方法，使得在推理过程中能够仅使用整数运算，显著提高了计算效率且降低了内存需求，推动了量化技术在实际应用中的发展，后续许多研究在此基础上进一步探索更细粒度的量化策略或优化整数运算的实现方式[18]。Hubara I 等学者聚焦于低精度算术在神经网络中的应用，分析了不同位宽量化对模型训练和推理的影响，研究表明适当的低精度量化不仅能减少计算资源消耗，还能通过特定的训练方法维持模型性能，引发了对低精度量化下模型优化算法改进的研究热潮，例如开发适用于低精度数据类型的梯度更新策略[19]。

知识蒸馏：利用大模型的 “dark knowledge” 作为监督信息来指导小模型训练，使小模型能够学习到更多知识，提升性能，达到模型压缩的效果。例如 Remero A 等人通过将教师模型的中间层特征作为提示信息传递给学生模型，帮助学生模型更好地学习[20]。Zhang C 等使用仅有编码器的知识蒸馏，将原始模型（教师模型）的图像编码器部分的知识转移到学生模型[21]。Zhou C 和 Li X 等学者通过多阶段训练即先进行编码器仅有的知识蒸馏再进行带有提示的循环知识蒸馏，克服任务无关的编码器知识蒸馏在处理框提示时效果不佳的局限性，并提出了一种新的知识蒸馏策略，即在蒸馏过程中引入提示，该方法不仅提升了模型对不同类型提示的响应准确性，还改善了困难区域的边界紧致性和结果准确性[22]。在一些研究中，卷积层和全连接层被压缩以减少 DNN 模型的处理时间[23,24,25,17]。

（3）计算图替换

计算图替换也是一种常见的模型轻量化方法。DFN 选择一组常见但不同的算

中国知网 https://www.cnki.net

子来对原模型中的算子进行替换[26]。然而，DFN 侧重于替换整个 DNN 模型，这不能直接推广到层级粒度。 LegoDNN 实现了块级 DNN 训练和缩放，它实现了逐层替换，然而，LegoDNN 是为具有单一类型处理器的平台设计的[27]。此外，它没有考虑跨处理器 DNN 推断的层之间的通信开销。Tensorflow、Pytorch 和 TVM[28]以预定义的顺序执行基于贪心规则的替换。它们使用的一个规则是用等效的融合算子替换卷积、批量归一化和激活函数，进而消除中间结果并减少内存访问。Zhao Y 等学者提出了一个统一的框架 AutoGraph，以获得有利于 GPU 内核并行执行的高度优化的计算图[29]。采用一种新的动态规划算法，结合回溯搜索，通过混合关键路径代价的快速性能估计，探索最优图优化解，并且利用 CUDA 图启动的基于 GPU 多流的精确运行时信息来确定优化的收敛性。

1.2.2 移动端神经网络处理器发展现状

（1）CPU-NPU 异构融合处理器的发展方面

随着人工智能在移动设备上的广泛应用，神经处理单元（NPU）也逐渐成为移动端处理器的重要组成部分。为了满足移动设备对深度学习技术的需求，神经处理单元（Neural network Processing Unit，NPU）架构开始受到关注。Jang J-W 等人提出了一种灵活高效的 NPU 架构，用于三星旗舰移动片上系统（SoC）[30]。通过设计高能效的内积引擎、可重构的 MAC 阵列、动态内部存储器端口分配和支持混合精度算术的高效架构，该 NPU 在执行 8 位量化的 Inception-v3 模型时，单个 NPU 核心可达到 290.7 FPS 和 13.6 TOPS/W 的性能。

华为在 2018 年推出昇腾 310，它是的人工智能片上系统（SoC），专为边缘计算产品和移动端设备设计，广泛应用于图像识别、视频分析、语音处理等 AI 应用场景。并且兼容多种主流深度学习框架，并且华为提供了完整的工具链和服务平台，如 MindSpore，使得开发者能够轻松部署模型到边缘端。

RK3588 是一款高性能应用处理器，由瑞芯微电子于 2021 年设计，面向 ARM PC、边缘计算和个人移动互联网设备。它内置高性能自研 RKNPU（Rockchip Neural network Processing Unit，以下简称 RKNPU）架构，支持 INT4/INT8/INT16/FP16/BF16/TF32 等多种数据类型，算力高达 6TOPS，特别适合复杂 AI 计算任务。

（2）NPU 性能提升方面

Park H 和 Kim S 等学者指出自 2017 年 NPU 集成到智能手机应用处理器以来，智能手机 NPU 的性能每年都在翻倍[31]。这表明移动端 CPU-NPU 处理器在计算性能方面取得了快速的发展。

中国知网 https://www.cnki.net

为了适应不同的深度学习应用，移动端 NPU 需要具备更高的灵活性和通用性。Jung H 等人提出了一些技术来最大化 NPU 核心的利用率并降低设备端推理的延迟[32]。通过将深度神经网络的层进行分区，在多核 NPU 上并行执行，并在单核内采用流水线执行，可以提高性能。文献还提出了新颖的优化技术，考虑了分区方向、执行顺序、同步和全局内存访问等因素，以缓解并行执行和中间张量加载与存储中的瓶颈问题。

与此同时，智能手机和机器人等边缘设备上部署先进的大型语言模型是一种日益增长的趋势，它增强了用户数据隐私和网络连接弹性，同时保留了智能功能。然而，这样的任务表现出单批计算，算术强度极低，这给有限的边缘资源带来了巨大的内存占用和带宽需求的重大挑战。为了解决这些问题，Yu Z 等学者提出了 Cambricon LLM，这是一种基于小芯片的混合架构，具有 NPU 和专用 NAND 闪存芯片，可实现 70B LLM 的高效设备内推理[33]。

1.2.3 异构融合处理协同计算技术研究现状

（1）算子级粒度并行策略

由于通信开销，层级调度可能会导致 DNN 推理的高开销。Kim Y 和 Kim J 等对层级粒度进行再细分，提出了层级分割[34]，使用参数 μ（0<= μ <=1）作为层分割的程度，将每一层按照各自的参数 μ 划分为两部分，一部分使用 CPU 推理，另一部分使用 GPU 推理，推理的结果进行合并后再按照参数划分，推理出最终结果，μ-layer 使用代表性 NN 和移动设备的实验表明，提高了设备上推理 69.6% 的速度和 58.1%的能量效率。Jia F 提出了 CoDL，提出 CPU 和 GPU 间的多类型数据共享机制，通过设计多维分割和算子链来减小数据共享开销[35]。CoDL 的多维数据分割基于提出的时延预测器，可以通过给定的输入算子设置、分割方案预测并行执行的总时延。基于预测的时延找到DNN模型每个算子的最优分割维度和分割比。Wang M等采用矩阵乘法划分和任务调度来缓解移动CPU上的任务分布不平衡[36]。

（2）层级粒度协同计算

Zhang L 等人将模型拆分成多个 kernel，每个 kernel 是 1 到若干个算子的组合，对每个 kernel 进行时延预测，最后得出总的时延预测时间，难点在于 kernel 的组合形式，即考虑哪一些算子发生了融合[37]。KANG W 等人提出了 LaLaRAND，这是一个实时的层级 DNN 调度框架，通过将 cpu 友好的量化与细粒度的 CPU/GPU 分配方案(一层资源每层)紧密耦合，同时在不影响时序保证的情况下减轻精度损失，从而实现单个 DNN 层的灵活 CPU/GPU 调度[38]。LalaRAND 将模型量化技术

中国知网 https://www.cnki.net

与细粒度的 CPU/GPU 资源分配相结合，以解决 CPU-GPU 平台上 DNN 推理的不对称性质。Tan T 等学者提出了最大精确度问题与最小时间问题，首先使用启发式算法寻找层组合来满足这两个问题，但缺点是只能搜索到少量的层组合，因此之后使用基于模型分区的机器学习方法，尝试了多种不同的先进机器学习模型，最后认为梯度提升回归树是最好的模型（因为平均绝对误差和平均绝对百分比误差最小且决定系数最高），实现了 DNN 模型将不同层分配给 CPU 或 NPU 资源的划分[39]。Ling N 等学者提出了一个新的系统名为 BlastNet，它利用了一种新的模型推理抽象——Duo-Block，来支持高效地跨 CPU 和 GPU 异构平台进行 DNN 模型推理。每个 Duo-Block 拥有双模型结构，使得能够在不同的处理器之间实现细粒度的推理任务分配，并且优化了计算特性和通信开销。为了生成这些 Duo-Block，BlastNet 使用了一种基于神经架构搜索（NAS）技术的算法，对 Duo-Block 的生成之初进行了优化[40]。

（3）边端异构处理器多 DNN 任务调度

在基于异构处理器的多模型场景下的计算资源调度方面，Xiang Y 等针对边缘计算和车载智能系统中多个DNN模型并发使用时遇到的响应时间不可预测以及系统资源利用率不足的问题，提出了一个名为 DART 的 DNN 任务调度框架，旨在为实时任务提供确定性的响应时间，并同时提高尽力而为任务的吞吐量[41]。DART 采用了一种基于流水线的数据并行 CPU/GPU 调度架构，其中异构的 CPU 和 GPU 被组织成不同并行级别的节点。该框架还包含流水线阶段设计和节点配置方案、准入控制、执行时间剖析以及运行时抢占调度技术。BlastNet 在运行时动态调度优化后的 Duo-Block，以提高 CPU 和 GPU 资源的利用率，可以在多个 DNN 模型并发执行时改善 CPU/GPU 的使用效率[40]。Hwang S 等人介绍了 GPU 基准测试程序的结果受处理器类型与所在集群影响，其中影响最小性能最好的是与硬件中断处理器同一个集群上的其他处理器[42]。

Lin C 等学者提出基于任务的算子并行性异步推理框架 TOP，TOP 结合了对任务到达模式对每个模型推理进度的影响的理解。它利用多智能体强化学习算法 MADDPG 来协同优化任务启动器和模型调度器，生成一对最佳的出队频率和计算图，可以通过增加单个模型的并发性或批量大小来显著提高其吞吐量，从而实现自加速[43]。后来 Lin C 提出了一种更细粒度的算子级跨处理器调度框架 COS，它可以更精确地为租户划分计算工作负载和切换开销，而无需修改或重新训练，并且 COS 在多租户场景中实现了比单处理器处理更高的吞吐量[44]。

在内存优化方面，Wang K 等人开发了 SwapNet，它将 DNN 划分为多个块，

中国知网 https://www.cnki.net

并按顺序换入和换出，以便大型 DNN 可以在较小的内存预算内执行，并且 SwapNet 消除了块交换过程中不必要的内存操作，同时保持与深度学习框架、GPU 后端和边缘 AI 设备的硬件架构的兼容性[45]。

1.2.4 现有研究现状的不足

（1）硬件适配性不足

当前面向移动端的模型优化加速虽然从硬件到软件的研究均有涉及，但大部分移动端的模型推理框架只限于单一类型的处理器。对于像 RK3588 这样含有 NPU 的异构处理器来说，现有的优化方法和工具链存在明显的局限性。轻量化模型的设计往往没有充分结合具体的硬件平台特性，导致即使是在 NPU 上运行时也不总是能够达到最优解。特别是对于一些输入或输出特征图是 5 维的算子，这类算子在 GPU 上有良好的支持，但在 RKNPU 上未被支持。此外，由于 RKNPU 的推理框架是闭源的，这限制了开发人员对算子在 RKNPU 上推理过程的改进能力。

（2）算子兼容性问题

在将模型转换为适合 NPU 运行的格式过程中，由于 NPU 对特定算子的支持度与速度的问题，某些算子可能会被自动替换为其他算子或发生算子融合。这种变化会导致原本基于算子链技术建立的时延预测器失效，因为这些预测器依赖于后端推理框架使用相同的算子。因此，当算子发生变化时，原有的性能预估就不再准确，给实际部署带来了不确定性。

（3）模型分区效率低

为了尽量缩小异构处理器之间的性能差异，研究人员尝试使用层级分区算法，并结合量化等方法来调整模型结构。然而，这种方法往往会带来一定程度的精度丢失，且最终的效果很大程度上取决于实际使用的硬件平台。对于 NPU 这样的处理器而言，其相对于 CPU 在处理某些特定类型算子上的性能优势并不能简单地通过量化手段来弥补。

最新的基于层级分区的 CPU-NPU 协同推理方法虽然引入了 GBRT（梯度提升回归树）来进行模型执行时间的预测，以寻找接近最优解的模型分区策略，但这需要进行大量的模型推理次数以获得足够的数据集用于训练 GBRT 模型。同时，该方法通过逐个将算子部署到不同的处理器上来获取推理时间并推算数据传输时延，这种方式会导致大量的部署时间成本，影响整体优化效率。更重要的是，上述工作主要集中在针对特定处理器优化深度神经网络（DNN）推理，难以直接应用于其他具有不同类型处理器的异构平台。

中国知网 https://www.cnki.net

1.3 本文的主要研究内容

本文的研究内容围绕 RK3588 异构融合处理器的特点及其现有问题展开，通过一系列创新性方法和技术，研究并实现一套深度学习模型部署技术以最大化利用 CPU 和 NPU 的计算资源，提高深度学习模型在移动端设备上的推理速度。图 1-2 为本文主要研究内容总体框架图，具体如下：

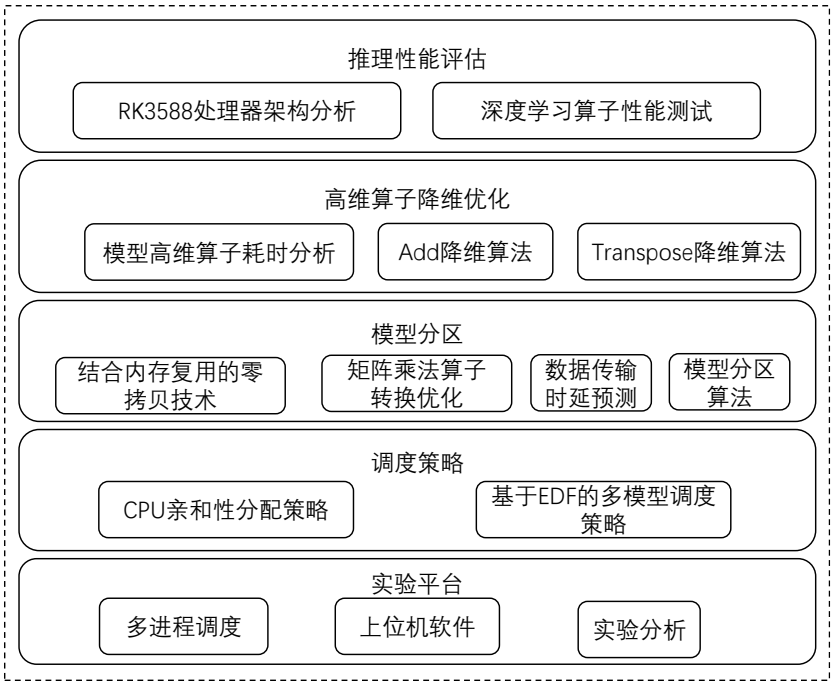


图 1-2 研究内容总体框架图

（1）通过全面而深入分析 RK3588 异构融合处理器硬件平台与两大模型后端推理框架，测试不同计算规模的各类算子发现现有的 CPU 与 NPU 推理框架潜藏的问题，进而为本课题的研究方向提供指导性方针。

（2）针对 Transformer 模型中存在超 4 维算子且 NPU 无法部署高维算子的问题，提出一种新颖的算子降维算法。通过对 5 维及以上的算子高维度来源进行溯源，找出造成算子形状为 5 维及以上的 Reshape 与 Transpose 算子，对其进行降维，实现 NPU 计算资源合理使用。

（3）针对 RK3588 上两种深度学习模型推理框架不能高效率使用计算资源计算某些特殊算子的问题，提出基于时延预测的模型分区算法，这部分包括通过零拷贝技术减少数据传输时间，使用内存复用技术减少模型内存占用，使用机器学习回归预测的方式建立数据传输时延预测模块，极大程度提高计算资源利用率和模型推理时间。

中国知网 https://www.cnki.net

（4）针对模型分区后 RKNN 模型转换过程中单独的 MatMul 算子前后可能出现的自动添加 Transpose 的问题，提出基于冗余算子组合的纯净矩阵乘法算子转换方法，实现 Transpose 算子与转换为 RKNN 格式的 MatMul 算子的完全分离。

（5）最后实现了基于 EDF 的多模型调度策略，减少任务的平均响应时间。

1.4 论文组织结构

第 1 章绪论部分，首先介绍了本课题的来源、研究背景和意义，接着依次介绍了移动端神经网络处理器发展现状、面向移动端的深度学习模型轻量化研究现状以及异构融合处理协同计算技术研究现状，并分析了现有研究现状的不足之处。最后阐述了本文的研究内容和组织结构。

第 2 章针对本课题使用的 RK3588 处理器进行全面而深入的分析，研究了两种分别适用于 CPU 与 NPU 的推理框架，测试并分析了不同计算规模的算子分别在 CPU 与 NPU 上的推理速度与数据传输时间，为后续进行模型简化与模型分区奠定了理论基础。

第 3 章针对 NPU 对不合理的算子顺序优化不完善和对高维算子支持度低的问题提出了两大类用于计算图优化的算法，第一是针对 RKNN 模型添加性能较低的 Transpose 的问题提出基于冗余算子组合的图等效变换算法；第二是针对 NPU 对高维算子支持度低的问题提出两类算子降维算法。

第 4 章针对 RK3588 上两种深度学习模型推理框架不能高效率使用计算资源的问题提出基于时延预测的模型分区算法和基于 EDF 的多模型调度策略，并进行了实验验证。

第 5 章介绍用于模型优化与部署的平台的设计与实现，包括硬件平台与软件系统。之后分别针对目前主流的 Transformer 类模型、卷积神经网络类模型和以全连接层为主的模型进行了测试，并得出最终的实验结果。

中国知网 https://www.cnki.net

第2章 面向深度学习的 RK3588 处理器性能分析

2.1 引言

RK3588 作为一款高性能的国产移动端处理器，集成了强大的 CPU 和 NPU，是实现移动端深度学习应用的理想平台。特别是 RK3588 具有的大小核架构与多核 NPU 架构，使得开发者可以在其之上实施灵活的模型优化技术。然而，现有的深度学习推理框架无法高效率地执行某些算子，例如使用 CPU 作为主要推理后端处理器的推理框架 ONNX Runtime（Open Neural Network Exchange Runtime，ORT）不能高效执行矩阵乘法算子与卷积算子，而使用 NPU 作为主要推理后端处理器的推理框架 RKNN Runtime（Rockchip Neural Network Runtime，RRT）不能高效执行维度交换算子与维度重塑算子，这使得一些含有特定算子的模型推理效率低下。

为了充分利用 RK3588 的特性，本章首先从深度学习的角度分析其 CPU 和 NPU 架构特点，接着对两种推理框架即 ORT 与 RRT 进行详尽研究，通过对比不同规模算子在两者上的表现，能够深入了解各自的优势和局限性，这为接下来的算子优化和模型分区策略提供了理论支持，最后基于以上研究提出总体优化方案。本章的研究不仅对于提高移动端深度学习应用的效率尤为重要，也为接下来的优化算法的设计与实现提供了理论基础。

2.2 RK3588 处理器架构概述

RK3588 是一款专为智能设备设计的高性能 SoC（System on Chip），它不仅搭载了四核 A76+四核 A55CPU 和 Mali-G610 GPU，还内置 3 核总共 6 TOPs 算力的 NPU，用于加速神经网络运算。本小节从深度学习算子的角度，通过分析 RK3588 上具体的 CPU 大小核性能差异与 NPU 架构特点，得出基于 RK3588 上 CPU 与 NPU 的深度学习任务优化方案。

2.2.1 CPU 架构

如图 2-1，RK3588 搭载四核 Cortex-A55 和四核 Cortex-A76 集群，其中所有核心均为单线程。Cortex-A55 处理器是一款中端、低功耗处理器。Cortex-A76 具有高性能、低功耗特点。因此，RK3588 的 CPU 属于典型的 ARM 大小核架构，而这种设计允许处理器根据任务负载动态调整工作核心，从而在性能和功耗之间取

中国知网 https://www.cnki.net

得平衡。

对于大核来说，其具有高主频与多线程处理能力：Cortex-A76 大核的工作频率高达 2.4GHz，能够提供强大的单线程和多线程计算能力。相比之下，Cortex-A55 小核以较低的主频运行（1.8GHz），更适合处理轻量级的任务。此外，大小核之间的高速缓存大小较大差异，大核的一级缓存容量为小核的两倍，二级缓存为小核的四倍，因此大核也更适合处理访存密集型任务，而当系统处于待机状态或者执行简单的进程时，使用小核可以减少不必要的电力消耗。

因此，在基于 CPU 大小核的深度学习任务部署的优化技术研究中，应该对大小核结构进行合理的使用，例如使用大核执行计算密集型任务如深度学习任务中的卷积与矩阵乘法等算子的计算，而小核可以用于响应系统中断，避免进程切换带来的不必要的性能开销。

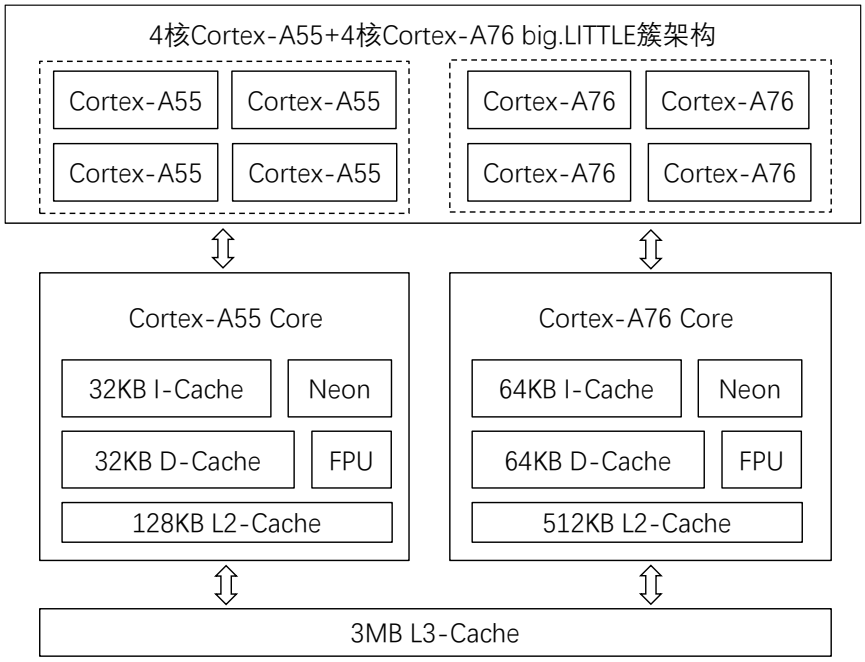


图 2-1 RK3588 CPU 框图

2.2.2 RKNPU 架构

（1）NPU 协同工作架构

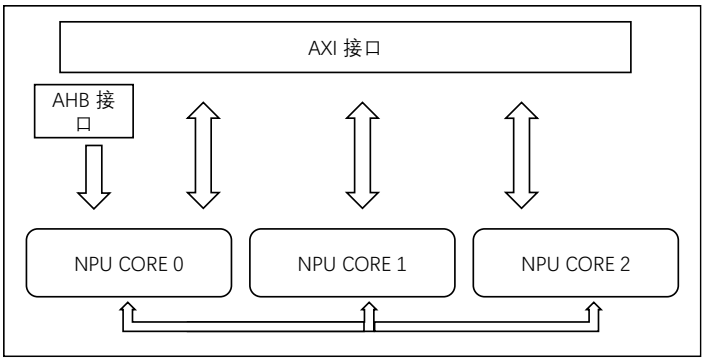
RKNPU（Rockchip Neural Processing Unit，以下简称 RKNPU）是 RK3588 处理器上专门用于神经网络的处理单元，它旨在加速人工智能领域(如计算机视觉和自然语言处理)的神经网络模型。如图 2-2(a)，RKNPU 架构包括三个 NPU 核心，它支持三核心协同工作、双核心协同工作和独立工作。先进高性能总线（Advanced High performance Bus，AHB）接口用于配置，仅支持单个 NPU 核心读取，而 AXI

中国知网 https://www.cnki.net

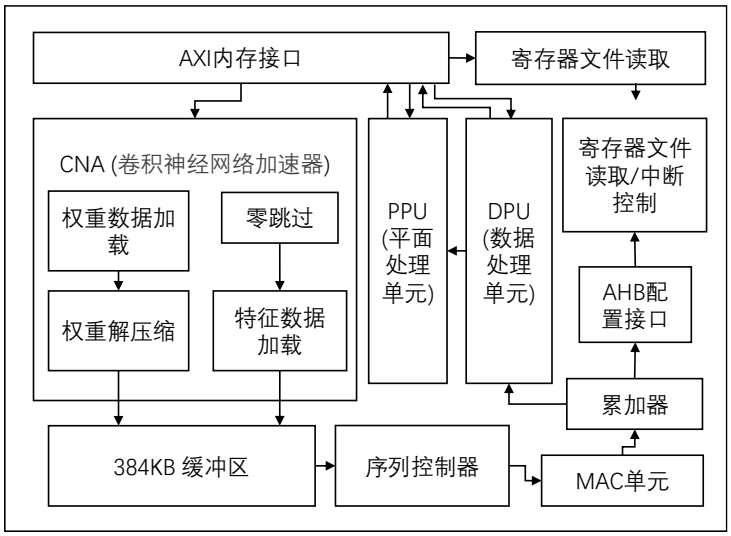
（Advanced eXtensible Interface）总线接口用于从存储器中获取数据，每一个 NPU 核心都可以与之交互。

（2）NPU Core 架构

如图 2-2(b)，NPU Core 包括多个单元。首先是 AHB/AXI 接口， 主接口 AXI 用于从内存中获取数据；从接口 AHB 用于访问和配置寄存器。而神经网络加速引擎是神经网络算术的主要处理模块，该模块包括卷积神经网络加速器（Convolution Neural Network Accelerator，CNA）、内部缓冲区、乘积累加运算（Multiply Accumulate ，MAC）单元和累加器。它提供了并行卷积的功能，并支持 INT4、INT8、INT16、FP16、BFLOAT16 和 TF16。数据处理单元（Data Processing Unit，DPU）主要处理单数据计算，如逐元素加法、激活函数。数据处理单元还提供 Softmax、转置、数据排布转换等功能。平面处理单元（Planar Processing Unit，PPU）主要作用是处理来自数据处理单元的输出数据，如平均池化、最大池化、最小池化等。



(a) NPU 核心协同工作框图



(b) NPU 核心

图 2-2 RK3588 NPU 架构

中国知网 https://www.cnki.net

NPU 主要是为运行传统的卷积神经网络（Convolutional Neural Network，CNN）而设计的，这是因为 CNA 核心功能是通过输入图像或特征数据以及相应的权重来执行卷积。CNA 输出可以定向到 DPU，在 DPU 中可以执行加法、乘法和 RELU 等逐元素运算。随后，DPU 的输出可以传送到 PPU，在 PPU 中执行最小、最大和平均池化等操作。此外还可以选择直接将数据传输到 DPU 或 PPU，而无需卷积步骤。除此之外，每个 NPU 核心各子拥有 384KB 的内部缓冲区，同时也对矩阵乘法有一定的优化，这意味着数据从内存读取后首先存放到该缓冲区等待被计算。

因此要研究面向 NPU 的模型优化技术，首先需要结合 NPU 核心中的计算单元对不同类型算子进行分类，其次通过分析不同类型算子性能上的差异得出优化的方向。

2.3 深度学习模型推理框架选择与分析

为了充分发挥 CPU 与 NPU 的优势，选择合适的推理框架至关重要。对于 CPU 而言，ONNX Runtime 是一个良好的选择，它经过了专门优化，可以最大限度地利用 CPU 并行性在移动设备上进行模型推理。而对于 NPU，Rockchip 的 RKNN-Toolki 允许开发者轻松地将模型转换为适合 NPU 执行的格式即 RKNN（Rockchip Neural Network），并且能够自动完成一些底层优化工作。通过对这两种推理框架的研究，可以更好地理解它们如何映射到不同的硬件组件上，以及它们各自在处理特定类型算子时的表现。

2.3.1 面向 CPU 的推理框架 ONNX Runtime

（1）ONNX 格式

ONNX[46]（Open Neural Network Exchange）是一种机器学习模型的表示格式，如图 2-3，它实现了跨平台和跨框架，多个深度学习框架如 Pytorch、Tensorflow、MXNet 支持导出 ONNX 格式模型，并且多种平台如基于 ARM 的移动端平台、基于 AMD 处理器的 PC 端平台均实现了 ONNX 的推理引擎。ONNX 拥有一套标准化的算子定义，其 API 简洁易懂，因此 ONNX 格式也成为当前最流行的中间格式，也使得 ONNX 模型容易被二次开发例如用于算子替换、维度修改、算子融合等。本文使用的 RK3588 处理器也对 ONNX 模型的部署和格式转换提供了良好的支持，因此使用 ONNX 作为模型使用 CPU 推理和转换为能在该处理器的 NPU 上模型的格式。

中国知网 https://www.cnki.net

不同于一般的图的节点与边的存储方式，ONNX 模型具有特殊的图存储结构，它自定义了一组算子与数据类型。ONNX 格式将模型定义为计算图[46]，它是一个有向无环图（Directed Acyclic Graph，DAG），描述了模型的结构，而计算图包含 Nodes、模型输入 Inputs、模型输出 Outputs 和初始化器 Initializers。每个节点表示一个算子，并且可以有多个输入和输出，节点拥有具体的依照 ONNX 规范命名的算子类型，ORT 后端推理执行器根据算子类型执行算子推理。ONNX 的计算图没有边的定义，而是用节点的输入输出张量来代替，每个张量拥有自身形状的描述、名称，因此每个节点保存了其输入张量与输出张量的名称，同一个张量可能对应于多个输入节点和多个输出节点。本文依照 ONNX 规范对 ONNX 模型进行修改以达到计算图优化目标。

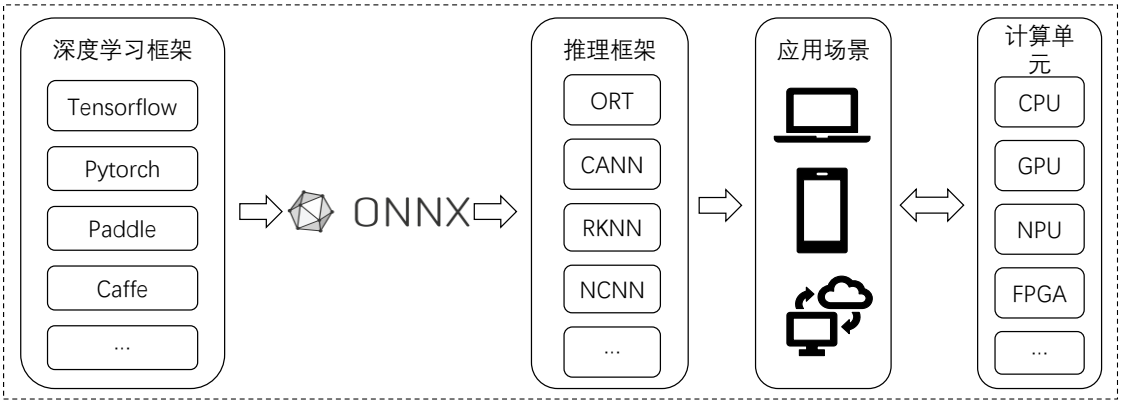


图 2-3 ONNX 框架功能

（2）ONNX Runtime

ONNX Runtime[47]（以下简称 ORT）是适配 ONNX 格式的跨平台高性能推理引擎。ORT 支持 Windows、Linux 和 macOS 等操作系统，可以使用 CPU、GPU 作为 ONNX 模型部署的后端，除此以外还支持其他加速器如华为 NPU。RK3588 使用的 A76 与 A55 CPU 均提供了 SIMD 指令集，ORT 可以利用这些底层硬件特性实现模型的高效推理。对于开发人员来说，由于 ORT 对于 ONNX 格式的良好支持、API 定义简洁、支持自定义算子，使得从训练到推理的迁移过程得到了简化。也因此本文中只需要获得 ONNX 格式的模型便可以方便地将模型部署至 RK3588 处理器。

ORT 的工作流程如图 2-4，ORT 通过模型加载器将 ONNX 模型加载进入内存(In-Memory Graph)，并且根据设置或默认配置进行图级别的算子优化，包括算子融合、算子替换。下一步，ORT 将计算图进行分区成为适用于后端注册器的可并行执行的子图集合，而后端由 ORT 使用者提交到注册器进行注册（可以使用 ORT

中国知网 https://www.cnki.net

自带的也可以是用户自定义）用于子图的推理。接着 ORT 读取输入数据经由并行化运行模块执行推理，在所有的子图按照依赖关系执行完毕以后输出结果。

用户实际使用 ORT 时需要创建 ORT 会话对象用于 ONNX 模型推理与获取推理结果。推理之前可以对 ORT 会话对象进行个性化的配置，例如通过配置选项参数可以控制模型的后端设备选择、并行度、模型预热、性能分析。ORT 其内部实现了高度的算子推理并行化体系，因此 ORT 并行度的控制对于模型推理的总体性能表现尤为重要。

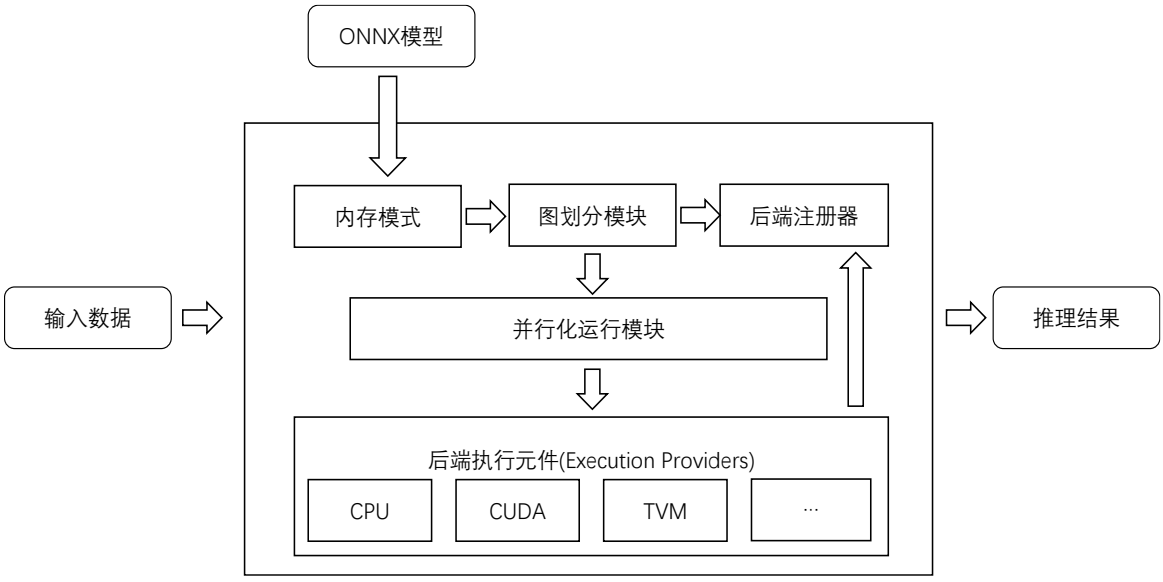


图 2-4 ONNX Runtime 工作流程

2.3.2 面向 RKNPU 的推理框架 RKNN Runtime

（1）RKNN 格式

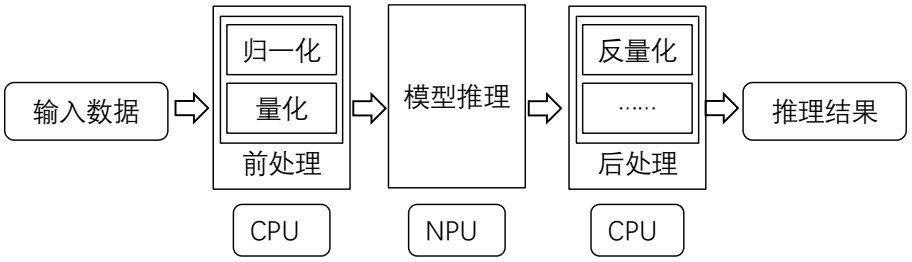
RKNN 是使用 RKNPU 的模型格式，由其他深度学习框架的模型格式转换而来，而由于对 ONNX 格式支持度的优势，最常用的是以 ONNX 为原模型的格式转换方式。RKNN-Toolkit2 是为用户提供在 PC 上进行模型转换、推理与性能评估的开发套件。本文通过 RKNN-Toolkit2 获取模型运行时各个算子具体的性能和内存评估情况，进而完成测试分析。

（2）RKNN 零拷贝

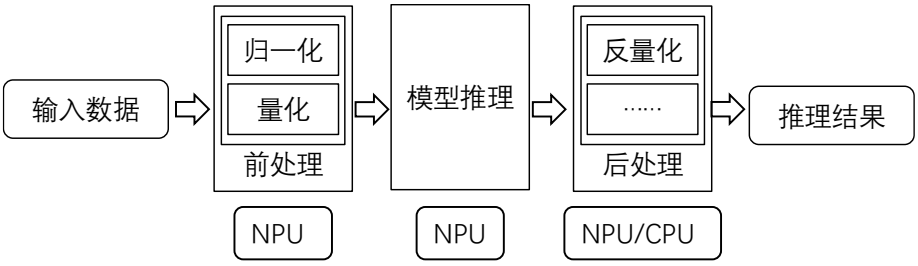
RKNN Runtime（以下简称 RRT）负责加载 RKNN 模型，并调用 NPU 驱动实现在 NPU 上推理 RKNN 模型。推理 RKNN 模型时，包括原始数据输入预处理、NPU 运行模型、输出后处理三项流程。根据不同模型输入格式和量化方式，RRT 提供两种处理流程分别为通用 API 和零拷贝 API。

中国知网 https://www.cnki.net

如图 2-5 为通用 API 和零拷贝 API 推理数据处理流程。通用 API 的优点为易于使用，其中对数据的归一化、量化、数据排布格式转换、反量化等均在 CPU 上运行，模型本身的推理在 NPU 上运行；而零拷贝 API 优化了通用 API 的数据处理流程，归一化、量化和模型推理都会在 NPU 上运行，NPU 输出的数据排布格式和反量化过程在 CPU 或者 NPU 上运行。零拷贝 API 对于输入数据流程的处理效率会比通用 API 高。支持数据在不同的 NPU 核心之间流动，没有数据拷贝，减少 CPU 及 DDR 带宽消耗。本文考虑到实际场景下的各个模型的规格，利用这两种 API 的特性减少了数据传输时间开销。



(a)通用 API



(b)零拷贝 API

图 2-5 RKNN Runtime 数据处理流程

（3）RKNN 内存复用

RKNPU 内的所有数据指针（例如输入、权重、输出、任务列表）均为 32 位，并且必须引用物理内存，因此一个 NPU 核心只能分配 4GB 大小内存，这造成了现有的一些大模型无法直接使用 RRT 进行部署的问题。但 RKNN API 提供了自定义 NPU 内存的机制，可以指定模型中间的特征图的内存由外部分配。因此，提出内存复用技术。当多个模型串行执行时，可以将同一块内存区域分配给它们，当前一个模型执行完毕，则从内存中清除它，转而将下一个模型加载进这块区域。

如图 2-6，若 Model_1 与 Model_2 顺序执行，则计算出二者需要的内存区域的较大者，分配该区域供二者使用。本文将这项内存复用技术用于模型的权重与中间特征图的内存分配，结合零拷贝技术在保持推理速度的同时降低模型所需内存，并用于模型分区后的推理调度。

中国知网 https://www.cnki.net

由上述可知，针对 NPU 的优化在内存优化方面可以从内存复用实现，在减少模型总体运行时间方面，可以使用零拷贝技术进行优化。

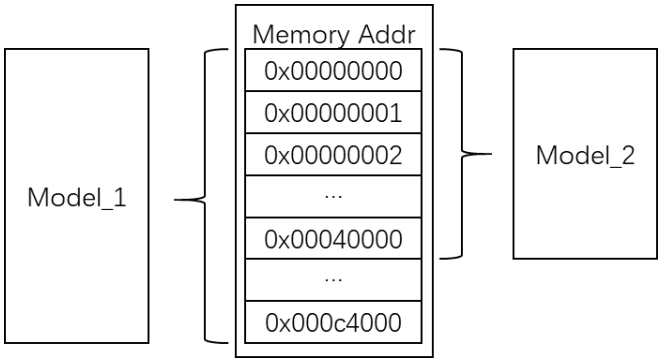


图 2-6 RKNN 内存复用

2.4 算子性能测试与分析

为了深入了解 CPU 与 NPU 在处理不同规模算子时的性能表现，进行了全面的测试。测试涵盖了从简单的基本算子到复杂组合算子的各种情况，包括但不限于卷积、矩阵乘法、激活函数等。每个算子或算子组合分别使用 CPU 或 NPU 单独执行，记录对应的推理时间和数据传输时间。实验结果显示，尽管 NPU 在处理卷积类算子方面表现出色，但在处理 Reshape 和 Transpose 时，由于算子维度过高或 NPU 对某些算子支持度低，CPU 反而可能成为更优的选择。这些问题的发现为接下来讨论的计算图优化与模型分区策略奠定了基础。

根据 2.2 小节对于 NPU 与 CPU 的数据处理流程分析，NPU 拥有专用于卷积计算、激活函数计算、平面数据计算等计算单元，结合 TVM 的算子的定义，可以将算子分为 Element-Wise 类[28]，卷积类，矩阵乘法类，数据搬运类，如表 2-1。

表 2-1　神经网络常见算子分类

算子类型	具体算子
Element-Wise	Add， Mul， Relu，LeakyRelu
卷积	Convolution
矩阵乘法类	MatMul
数据搬运类	Reshape，Transpose

接下来对不同规模的算子进行测试并进行分析。具体测试的软硬件环境配置如表 2-2 所示。特别地，在搭载 8 核 CPU 与 3 核 NPU 的 RK3588 EVB 开发板上，针对 CPU 使用 ORT 开启最大线程（即 8 线程）部署，针对 NPU，使用 RRT 开启 NPU 三核并行模式部署。

中国知网 https://www.cnki.net

表 2-2　算子性能测试环境配置

名称	具体配置
CPU	4×Cortex-A76、4×Cortex-A55
NPU	3×NPU Core
推理框架	ONNX Runtime，RKNN Runtime

2.4.1 CPU 与 NPU 推理时间对比

（1）Element-Wise 类算子

逐元素计算类算子，也称为 Element-Wise Operators 或逐元素操作，是指那些对张量（Tensor）中每个位置上的元素独立进行相同运算的算子。这意味着，对于两个或多个输入张量，它们相应位置上的元素将一一配对，并且在这些配对元素上执行相同的数学或逻辑运算，生成一个新的输出张量，其形状与输入张量相同（假定输入张量具有相同的形状；如果形状不同，则需要遵循广播规则）。这类算子不会改变数据的维度或形状，只改变各个位置上的数值。

如图 2-7 为使用 ORT 和 RRT 推理 Element-Wise 类算子的时间统计，测试包含 Add、LeakyRelu、Mul、Relu 四类不同的算子，从图表中可以看出，不同算子在较小的输入规模（如 1x32x100x100）下，无论是 CPU 还是 NPU，所有算子的延迟都非常低，几乎可以忽略不计。在数据规模在 1x128x100x100 时出现了 CPU 推理时间优于 NPU 的情况。当输入规模增加到 1x128x200x200 时，延迟开始显著增加，在这个规模下，NPU 对比 CPU 的优势开始显示出来，尤其是在 Mul 和 Relu 算子上。随着输入规模的增大（如 1x512x400x400），对于较大的输入规模，NPU 的性能优势更加明显。在 1x512x400x400 规模下，NPU 的延迟显著低于 CPU，因此在处理大规模数据时，使用 NPU 能够显著提高计算效率，减少延迟时间。

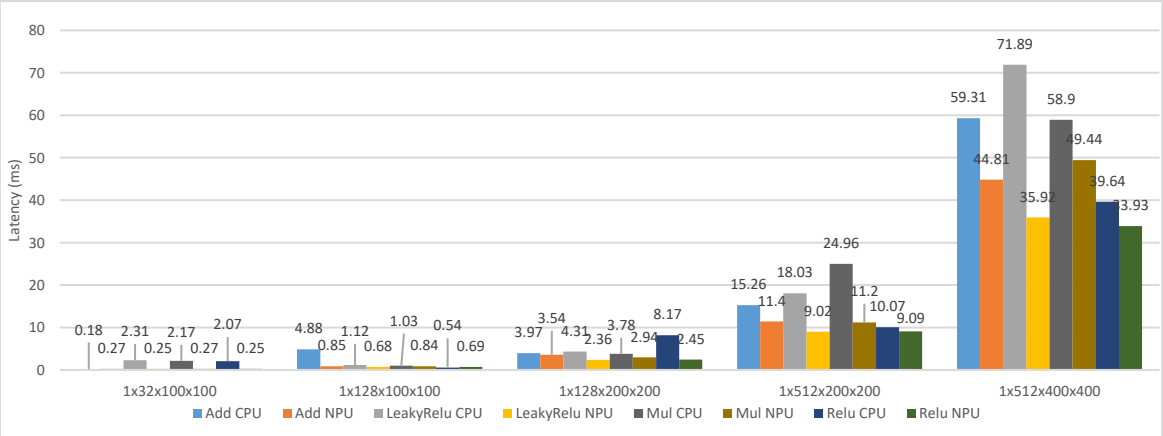


图 2-7 Element-Wise 类推理时间

中国知网　https://www.cnki.net

（2）卷积类算子

Conv2d（二维卷积）是一种在深度学习中广泛使用的操作，特别是在处理图像数据时。它通过应用一个或多个可学习的滤波器（也称为内核或卷积核）来提取输入特征图中的局部特征。对于一个输入张量 X 和一个卷积核 K，输出张量 Y 的计算可以用公式(2-1)表示：

$$Y = \sum_{c_{in}=0}^{C_{in}-1} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} X(n, c_{in}, i+m, j+n) \cdot K(c_{out}, c_{in}, m, n) + b \quad (2\text{-}1)$$

其中 X 是一个四维张量，形状为 $[N, C_{in}, H_{in}, W_{in}]$，分别代表批次大小、通道数、高度和宽度，是卷积核（滤波器），K 是一个四维张量，形状为 $[C_{out}, C_{in}, K_h, K_w]$，分别代表输出通道数、输入通道数、卷积核的高度和宽度。

其输出张量维度将为$[N，\ C_{out}, H_{out}, W_{out}]$，其中$H_{out}, W_{out}$的大小由公式(2-2)给出：

$$H_{out} = \left\lfloor \frac{H_{in}+2\times padding-dilation\times(K_h-1)-1}{stride} \right\rfloor + 1 \quad (2\text{-}2)$$

$$W_{out} = \left\lfloor \frac{W_{in}+2\times padding-dilation\times(K_w-1)-1}{stride} \right\rfloor + 1 \quad (2\text{-}3)$$

其中 stride 是卷积操作的步长，默认值为 1。padding 是在输入边缘添加的零填充，默认值为 0。dilation 是膨胀率，默认值为 1，用于控制卷积核元素之间的间距。

由以上式子可以得知，每个输出元素需要进行$C_{in} \times K_h \times K_w$次乘法和相同数量的加法的操作，由于输出张量元素数量为$N \times C_{out} \times H_{out} \times W_{out}$，卷积算子的计算量（Floating Point Operations，FLOPs）可以用公式（2-4）表示：

$$FLOPs = 2 \times N \times C_{out} \times H_{out} \times W_{out} \times C_{in} \times K_h \times K_w \quad (2\text{-}4)$$

如图 2-8，横轴为使用公式(2-4)计算出不同规模的卷积计算的计算量，纵轴为使用两种不同推理框架（ORT 和 RRT）处理 Conv2d 操作时的时延。总体来说 RRT 框架的延迟时间显著低于 ORT 框架。ORT 框架在相似计算量下的推理时间波动较大。在较低计算量的情况下，ORT 和 RRT 的延迟时间都相对较低，但 RRT 的延迟始终低于 ORT。当计算量进一步增加，ORT 的延迟时间开始显著增加，在高计算量下（如 353894400），ONNX 的延迟时间达到了峰值，为 18.28ms，而 RRT 的延迟时间仍然保持在较低水平，为 1.30ms。

因此对于需要高效处理高计算量卷积算子的应用场景，使用 RRT 推理框架更加适合，因为它能够在保证性能的同时提供更低的延迟时间。

中国知网 https://www.cnki.net

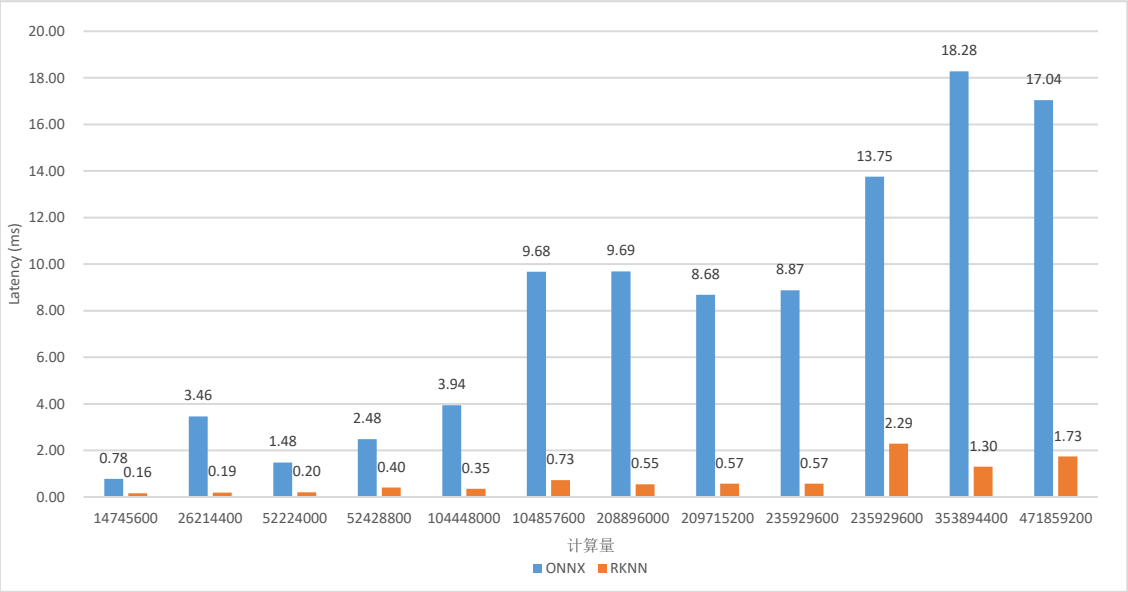


图 2-8 卷积类推理时间

（3）矩阵乘法类

如图 2-9 为矩阵乘法算子部署在 CPU 与 NPU 的推理时间的对比，在所有测试的输入规模下，NPU 的延迟时间均低于 CPU。在小规模输入下，CPU 和 NPU 的延迟都非常低，几乎可以忽略不计， CPU 和 NPU 之间的性能差异不大。中等规模输入（例如 1x128x200x200），CPU 的延迟开始显著增加，为 19.28ms，NPU 的延迟也有所增加，但仍然较低，为 14.7ms。大规模输入下 CPU 的延迟显著增加，在输入规模为（1x512x400x400）时为 588.09ms，而此时 NPU 的延迟仅为 252.1ms，不足 CPU 的一半。

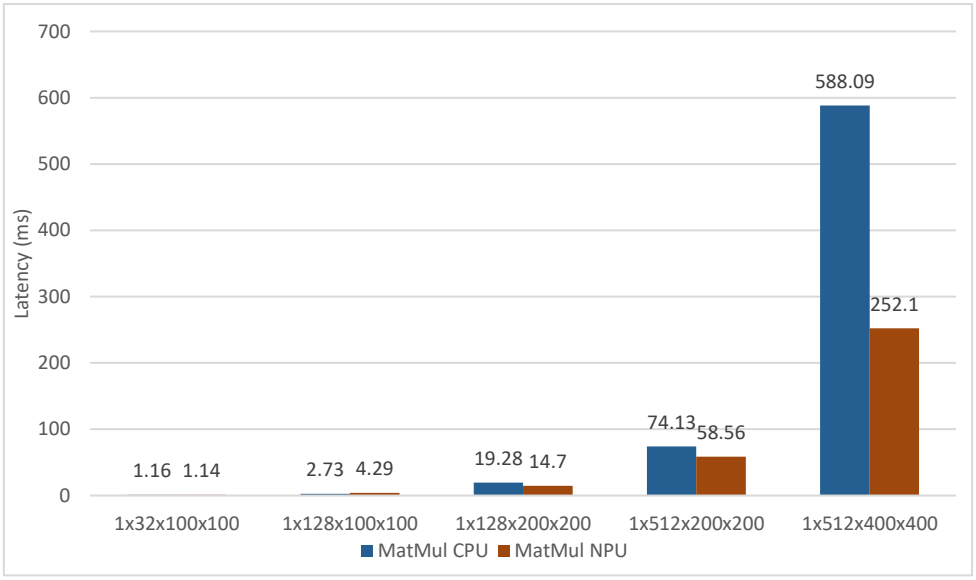


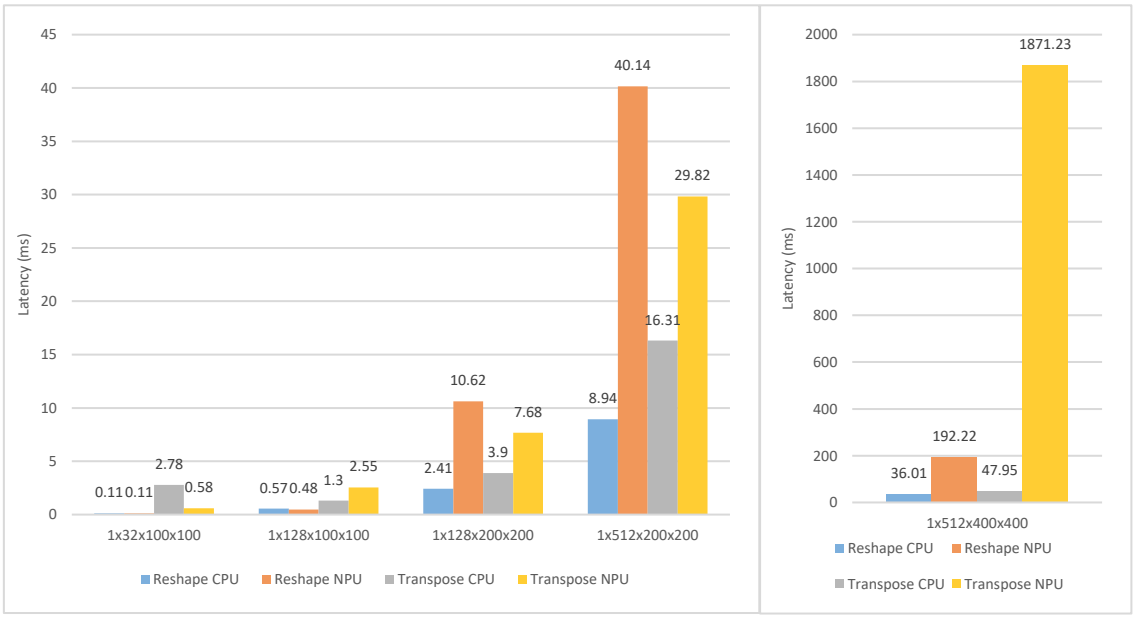
图 2-9 矩阵乘法类推理时间

中国知网 https://www.cnki.net

对于需要高效处理大规模数据的应用场景，NPU 的性能明显优于 CPU，能够提供更稳定的性能和更低的延迟时间，因此这种情况更适合 NPU。

（4）数据搬运类

如图 2-10 是数据搬运类推理时间对比，其中图(a)展示了不同输入规模下的 Reshape 和转置 Transpose 操作在 CPU 和 NPU 上的延迟时间，而图(b)则专门针对最大输入规模 1x512x400x400 的情况进行了详细对比。



(a) 中低规模下时延对比　　(b)大规模下时延对比

图 2-10 数据搬运类推理时间

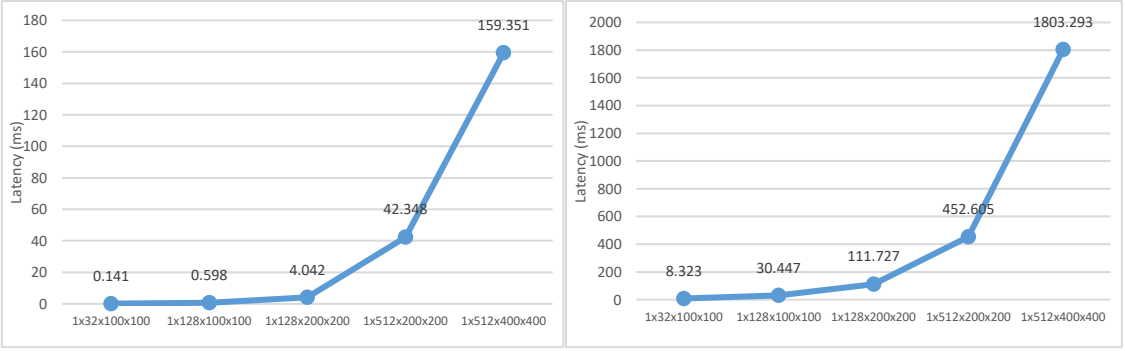
由图可知，在小规模输入下，CPU 和 NPU 之间的性能差异不大，而中等规模下（如 1x128x200x200），NPU 的劣势开始显现出来，对于 Reshape 来说，CPU 延迟为 2.41ms，NPU 延迟为 10.62ms，超过 CPU 的 4 倍。对于 Transpose 来说，CPU 延迟为 3.9ms，NPU 延迟为 7.68ms，NPU 约为 CPU 的 2 倍。而随着规模增大，CPU 的优势愈加明显，在数据规模达到 1x512x200x200 时，对于 Reshape 来说 NPU 推理时间超过 CPU 的 4 倍，而对于 Transpose，NPU 推理时间接近 CPU 的 2 倍。如图(b)，在大数据规模下，NPU 由于未适配这样规模的 Transpose 算子，所以将 Transpose 算子分配给 CPU 执行，由于数据规模太大造成了大量的数据传输时间开销，因此时延非常高，这对于深度学习任务来说是不可接受的。

因此，在执行大规模的 Reshape 与 Transpose 算子时，应尽量选择 CPU 作为模型推理执行器，而对于小规模的 Reshape 与 Transpose 算子，CPU 和 NPU 的性能相当，从节省功耗的角度来说，可以尽量选择 NPU 进行部署。

中国知网 https://www.cnki.net

2.4.2 CPU 与 NPU 数据传输时间对比

一般来说数据传输时间不与算子类型有关，而与 DDR 频率、数据规模、Cache 大小或缓冲区大小等因素相关，因此以 Relu 算子为例，对不同数据规模的进行测试。测试结果如图 2-11，对比两种不同推理框架下的数据传输时间，在所有数据量范围内，CPU 的数据传输延迟显著低于 NPU 的数据传输延迟。CPU 与 NPU 的数据传输延迟随着数据量的增加而显著增加，并且在较高数据量时，NPU 与内存之间的数据传输时延远高于 CPU 与内存之间的数据传输时延。



(a)CPU 数据传输时延　　(b) NPU 数据传输时延

图 2-11 CPU、NPU 与内存之间数据传输时延对比

因此，大规模输入与输出数据的情况下，数据传输时延可能成为模型性能的瓶颈，为此可以采取零拷贝的方法减小数据传输开销。具体的方案将在第 4 章详细说明。

2.5 面向 RK3588 的深度学习推理优化方案

通过 2.4 节的实验结果可以得知，NPU 的计算单元针对深度学习模型中的卷积层、全连接层等密集计算部分进行了优化，这样的硬件配置使得 RK3588 能够在保证低功耗的同时，提供卓越的计算能力。而通过 2.2 节的分析，相较于小核，在执行深度学习算法时，特别是那些涉及大量矩阵运算的操作如卷积层、全连接层等，大核可以显著加快模型推理的速度。而小核更适合轻量级任务，如处理系统中断、运行后台进程。而 CPU 则更适合处理复杂的控制逻辑和其他非计算密集型任务如矩阵拼接、维度转换、转置等算子。

因此，在构建异构算力平台时，充分利用两者的特性是提高整体效率的关键。针对 CPU 与 NPU 的差异性，提出模型分区策略几乎是一件必然的工作，然而，模型分区带来了两大新的挑战：一是多个子模型之间数据传输将带来巨大开销；二是有一些算子转换为 RKNN 格式后可能变得低效（将在第 4 章详细说明），需

中国知网 https://www.cnki.net

要找到有效的优化方法。本文将在后续的章节中展开以下工作：

（1）针对大小核差异性，提出 CPU 亲和性绑定策略，通过亲和性分配的方式将应用NPU核心与CPU小核绑定进而消除NPU中断处理的CPU核心切换开销，并最大程度利用大核进行模型推理，达到降低平均响应时间的目的。

（2）针对特定算子在 CPU 与 NPU 上推理时间的差异性，将一部分算子使用 CPU 部署，一部分使用 NPU 部署，从而提高模型整体推理速度。

（3）针对数据传输时间开销大的问题，结合零拷贝技术降低模型分区带来的数据传输时延。

（4）针对模型分区后出现多模型运行的场景，提出基于 EDF（Earliest Deadline First，最早截止时间优先）的多模型调度策略。

2.6 本章小结

本章从深度学习的角度探讨了基于RK3588异构融合处理器中的CPU-NPU架构。通过深入剖析 RK3588 的硬件特性，以及对两种主要推理框架——ONNX Runtime 和 RKNN-Toolkit 的研究，明确了它们在不同应用场景下的优势。最后，通过一系列算子级别的测试，得出了 CPU 与 NPU 在处理各种类型算子运行时的性能差异。这为后续章节中提出的优化方法提供了坚实的数据支持，同时也展示了异构融合处理器在未来移动端 AI 应用中的巨大潜力。

中国知网 https://www.cnki.net

第3章 面向 RKNPU 的高维算子降维算法研究

3.1 引言

在第 2 章的实验分析中了解到 RK3588 NPU 在加速深度学习中的矩阵乘法与卷积算子推理方面具有显著优势。然而，当前 NPU 也存在一些限制其性能发挥的问题，如对 5 维或更高维度算子的支持不足，以及特定算子（例如 Reshape 和 Transpose）在 NPU 上的执行速度远远低于 CPU。这些问题在处理复杂模型如 Transformer 时尤为明显。

如图 3-1 为本章总体优化路线图，本章通过对 Transformer 类模型的性能瓶颈进行详细分析，发现模型中的一些高维算子（如 5 维及以上的算子）以及低效的算子是主要的性能障碍。为应对这一挑战，本章提出了两种面向 NPU 的 Transformer 算子降维优化算法。首先，通过降低高维算子的维度，最大限度地防止 RRT 将这些算子迁移至 CPU，从而确保它们可以在 NPU 上高效执行。

通过实施这两种优化算法，成功提升了 Transformer 模型在 RK3588 NPU 上的推理性能，减少了从 NPU 到 CPU 的算子迁移现象，整体运算效率得到了显著提高。这些实验结果为后续章节实现模型分区提供了坚实的理论基础。

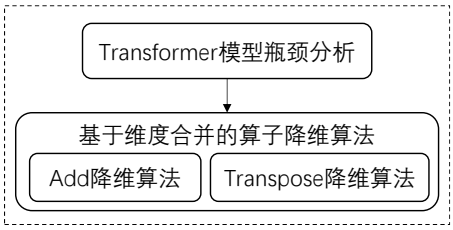


图 3-1 总体优化路线图

3.2 模型高维算子耗时分析

（1）Transformer 模型计算图算子类型分析

头盔中使用的图像分割应用所用到的 SAM_b[48]（Segment Anything Base）模型是基于 Transformer 的模型。Transformer 模型是一种深度学习模型，在 2017 年首次由 Ashish Vaswani 等学者提出[49]，它完全基于注意力机制，而不使用递归和卷积，在常规的 GPU 上更容易并行化。

如表 3-1 所示，在 Transformer 模型中，算子根据其功能可分为五大类：输入处理算子、注意力机制算子、编码器层算子、解码器层算子以及输出处理算子。

中国知网 https://www.cnki.net

首先，输入处理算子如 Gather（用于嵌入）和 Add（用于位置编码）负责将原始输入转化为包含位置信息的连续向量。接下来，注意力机制算子包括用于生成查询、键和值的 MatMul 和 Add，以及用于计算注意力权重的 MatMul、Div、Softmax 和 MatMul。编码器层算子则包含了用于稳定训练过程的 Add 和 LayerNormalization，还有用于非线性特征提取的 MatMul、Add 和激活函数（如 Relu 或 Gelu）。解码器层算子除了具有类似的结构外，还特别加入了掩码多头自注意力（涉及 Split、Concat、Reshape、Transpose 和 Where）和编码器-解码器注意力（同样使用 MatMul、Div、Softmax 和 MatMul），以确保解码过程中正确利用历史信息和编码器输出。最后，输出处理算子通过 MatMul、Add 和 Softmax 将解码器的输出映射到目标输出空间，完成从输入到输出的完整转换过程。

由以上分析可以得知，在 ONNX 模型计算图的表示中，Transformer 类模型主要由大量的 Element-Wise 算子如 Add、Div 与 Relu、矩阵乘法算子 MatMul、数据搬运类算子如 Transpose 和 Reshape 等构成。

表 3-1　Transformer 模型算子分类

分类	映射的具体 ONNX 算子
输入处理算子	Gather，Add
注意力机制算子	MatMul，Add，Div，Softmax
编码器层算子	Add，LayerNormalization，Relu，Gelu
解码器层算子 1	Split，Concat，Reshape，Transpose
解码器层算子 2	MatMul，Div，Softmax，MatMul
输出处理算子	MatMul，Add，Softmax
主要算子	MatMul，Transpose，Reshape，Element-Wise

（2）图像分割模型性能瓶颈分析

由上述可知，Transformer 类中主要含有大量 Element-Wise 算子、数据搬运类算子、矩阵乘法算子，而由第 2 章的算子性能分析可知，对于 MatMul 算子，规模较大时 NPU 性能远超 CPU，而对于 Transpose 与 Reshape 算子，规模较大时情况刚好相反。对于 Element-Wise 算子则两种计算资源的计算效率相当，NPU 略好一些。因此无论将 Transformer 模型直接使用 ORT 部署还是使用 RKNN 部署，都势必会有明显的性能瓶颈。

如表 3-2 显示了使用在 RK3588 EVB 开发板上使用 RRT 部署 SAM_b 模型中各种算子排行前五的推理时间，其中 Add 算子超过 99%的计算开销用于 CPU 计算，这因为在该 Transfomer 模型种 Add 是 5 维算子，所以被 RRT 分配到 CPU 上执行，然而这会造成大量时间传输开销，并且 RRT 只实现了单核 CPU 的算子推理，因此

中国知网 https://www.cnki.net

总时间占比高达 77.91%。此时 SAM_b 模型总推理时间为 136.244s。而 Transpose 算子也大量使用 CPU 执行，一部分原因也是有大量 Transpose 是 5 维算子的存在，还有一部分原因是。因此提出两种基于维度合并的降维算法分别用于 Add 算子与 Transpose 算子降维。

表 3-2　SAM_b 模型优化前各层推理时间

算子类型	数量	CPUTime(us)	NPUTime(us)	TotalTime(us)	TimeRatio(%)
Add	33	106140329	4433	106144762	77.91%
Transpose	148	20475460	1289881	21765341	15.98%
exSoftmax13	12	3050922	4349124	4932388	3.62%
Reshape	152	3105314	198269	1317045	0.97%
exMatMul	48	0	1240898	1240898	0.91%

3.3 RKNN 算子定义

（1）多维转置算子 Transpose

多维数组（也称为张量）的转置操作扩展了矩阵转置的概念，允许对更高维度的数据进行维度间的交换。在机器学习和深度学习中，尤其是处理图像、视频或其他具有多个维度的数据时，多维转置是非常常见的操作。ONNX 模型与 RKNN 模型对于多维转置算子 Transpose 定义如下：对于一个多维数组 A 来说，其形状可以表示为一个 n 元组$dims = (d_1, d_2, \dots, d_n)$，其中 n 是数组的秩（即维度数量），而 d_i表示第 i 维的大小。多维转置操作涉及到重新排列这些维度的顺序，从而生成一个新的张量 B，它的元素与原张量相同，但维度顺序不同。新顺序可以通过一个包含所有维度索引的列表$p = [p_1, p_2, \dots, p_n]$来指定，其中$P_i$是 A 的第 i 个维度在 B 中的新位置，使得$B[i_1, i_2, \dots, i_n] = A[j_1, j_2, \dots, j_n]$，其中$j_k$对应于$p_k$ 指定的位置，即j_k是 A 的第P_k维度上的索引值。

（2）RKNN 矩阵乘法算子 exMatMul

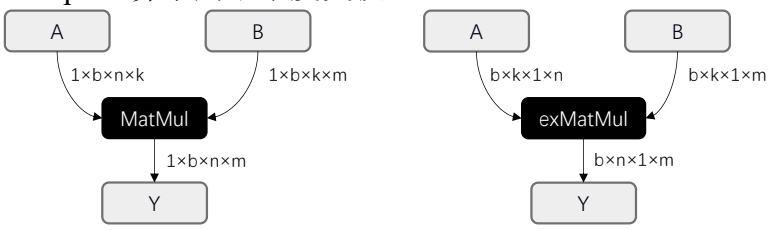
RKNN 与 ONNX 中对矩阵乘法定义由公式（3-1）给出：

$$Y = \begin{cases} A \cdot B + C, if\ C_{type} =' add' \\ A \cdot B + C, if\ C_{type} =' mul' \end{cases} \tag{3-1}$$

如图 3-2(a)，ONNX 模型中矩阵乘法算子双输入特征图两个输入特征图 A 和 B 的形状分别为[1, b, n, k]和[1, b, k, m]，输出特征图 Y 的形状为[1, b, n, m]，而为了适配 NPU 的计算，如图 3-2(b)，RKNN 实现的矩阵乘法算子 exMatMul 的两个输入特征图 A 和 B 的形状分别为[b, k, 1, n]和[b, k, 1, m]，输出特征图 Y 的形状为[b, n, 1, m]。因此若将只包含 MatMul 算子的 ONNX 模型转为 RKNN 模型之后，exMatMul

中国知网　https://www.cnki.net

前后将会添加 Transpose 算子用于维度交换。

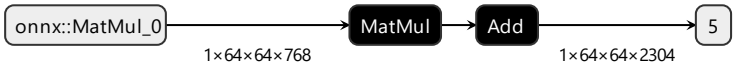


(a) ONNX 模型 MatMul 算子　　(b)RKNN 模型 exMatMul 算子

图 3-2 两类模型格式矩阵乘法算子表示

（3）RKNN 中的 1x1 卷积算子 Convolution

卷积是常用的用于提取图像局部特征的算子，而卷积核大小为 1x1 的卷积算子可以用于全连接层算子的等效替换。如图 3-3，ONNX 模型中全连接层被表示为一个矩阵乘法算子与一个 Add 算子的组合，而该全连接层组合在会被转换 RKNN 格式后会被以 Transpose-Convolution-Transpose 算子组合的形式进行保存与推理，其中卷积算子的核函数的形状为 1x1，其输入特征图 A 的形状为[1, b, n, k]，存于 Initializer 中的固定参数 B 的形状为[k, m]，C（可选）的形状为[1, b, n, m]；



(a) ONNX 模型全连接层



(b) RKNN 模型全连接层

图 3-3 全连接层的表达方式

（4）有向无环图

无论 ONNX 模型还是 RKNN 模型均可以将计算图抽象为一张有向无环图（Directed Acyclic Graph，DAG）。DAG 是图论中的一种图类型，特点是由节点和有向边构成，且没有环路，因此可以对节点按照拓扑序列进行排序。在深度学习领域中，模型被抽象为一个 DAG，其中算子作为节点，算子之间的依赖关系组成了有向边集。基于这一点，以下提出了模型修改的算法。

3.4 基于维度合并的高维算子降维算法

在 Transformer 模型部署过程中，常会出现一些算子处于 NPU 支持清单中但是无法使用 NPU 执行的问题，这是因为这些算子维度超出 NPU 可计算维度的定义范围，例如图 3-4(a)中的 Reshape 和 Add 算子，由于输入或者输出特征图的维度超过 4 维，所以 RRT 将它们分配到 CPU 上执行，由于数据传输开销与 CPU 单核

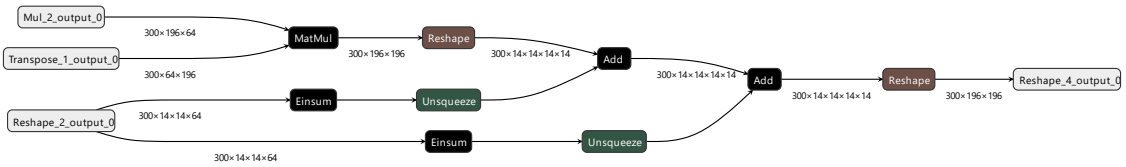
中国知网 https://www.cnki.net

推理，这会造成极大的时延，然而如果直接使用 ORT 只需要 1%的耗时。为了使这些算子能够部署到 NPU 上，本小节研究了两种基于维度合并的算子降维算法，这些算法不需要重训练，可直接应用于预训练模型。

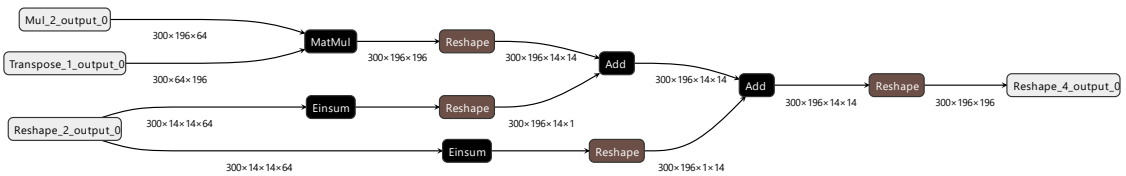
3.4.1 基于字典索引搜索的 Add 算子降维算法

Reshape 和 Unsqueeze 是深度学习框架中用于特征图形状操作的两个重要函数，ONNX 模型中也对这两种算子进行了实现。Reshape 算子的作用是在不改变输入数据和元素数量的情况下，返回一个具有指定形状的特征图，常用于展平或重构例如将多维特征图转换为一维向量，或将一维向量重新组织成多维特征图；而 Unsqueeze 算子的作用是在指定位置插入一个新的单一维度，即大小为 1 的维度，这不会增加特征图中的元素数量，但会改变其形状表示，从而影响后续操作的行为，常用于通过增加维度，可以使特征图更好地与其他特征图进行广播操作。因此这两类算子都可以灵活地调整特征图的结构以适应不同的运算需求。

如图 3-4(a)，模型中有时候会存在一些 5 维 Add 算子，而其主要来源是前置算子中有 Reshape 算子将低于或等于 4 维的特征图转换为 5 维再进行运算，而 Unsqueeze 算子在这些模型中插入维度的位置往往在第 3 维之后，用于 Add 算子的广播操作。在这个例子中，最后将由 Reshape 算子输出 3 维特征图，因此对于 Add 算子来说前 3 维的形状并不会影响最终输出的结果。同理，可以统计出影响 Add 算子输入维度的根源算子，在这个例子中是 Reshape 和两个 Unsqueeze 算子，考虑合并不受影响的维度统计各个由此提出基于字典索引搜索的 Add 算子降维算法。经由此算法后可将 5 维 Add 算子转换为图 3-4(b)中的 4 维算子，从而 Add 算子可以在 NPU 上运行。



(a)降维前



(b)降维后

图 3-4 Add 算子降维

中国知网 https://www.cnki.net

这里需要注意的是模型中往往存在某些算子尺寸规模相似，对齐与非对齐的运行耗时差别巨大的情况，差别的原因在于硬件对于部分非对齐尺寸的算子会需要额外的冗余操作来保证正确性，因此算子的尺寸设计对于模型性能也能起到很大的影响。在一些对齐尺寸下，除 NPU 运算效率更高外，对于 DDR 的读写也更友好，同等带宽条件下，更友好的读写会提升 DDR 的带宽效率，从而达到更好的性能。对于 RK3588 来说，对于 int8 类型，输入的通道数需要尽量为 8 的倍数，对于 float16 类型，输入的通道数需要尽量为 4 的倍数。并且同等规模的算子，Width 大 Height 小的尺寸，面向 DDR 读写更友好。因此尽量将第 2 维与第 3 维合并，可以最大限度使合并后的输入通道数能被 4 整除。

算法 3-1 是完整的基于字典索引搜索的 Add 算子降维算法，算法完整流程如下：

（1）首先为了快速索引 ONNX 模型中各算子的输出与输入边的，使用字典索引的方式存储。

（2）接着按照拓扑序列遍历计算图中的所有算子节点，若遇到输出形状维度大于 4 维的 Reshape 算子且未被遍历过，则创建字典用于保存将会该算子和搜索到的所有 Reshape 和 Unsqueeze 算子。

（3）进入递归查看输出边连接的算子是否是 Add，若不是则跳过，若是则向前溯源，直到溯源遇到了非 Add 算子。向前溯源是为了确保所有输入都是 Unsqueeze 或 Reshape，否则不可以用该方法降维 Add 算子。

（4）接着向后搜索直到遇到 Reshape 算子，完成搜索，将字典中算子进行降维。

（5）算子降维：遍历字典中所有的 Reshape 和 Unsqueeze 算子，将这些算子的任意连续两个同样位置维度都相等的维度合并，将所有的 Unsqueeze 算子改为 Reshape 算子。

算法 3-1：基于字典索引搜索的 Add 算子降维算法

输入：$G=(V,E)$，G是原模型计算图，V是算子节点集合，E是算子之间依赖关系

输出：G，针对 Add 进行降维之后的计算图G

```
function reduce_dimensions(G = (V, E)):  // 对 Add 算子进行降维
    for node ∈ V:  // 第一次遍历模型
        // 遇到 Reshape 算子且未被遍历过
        if node.type = 'Reshape' and visited(node) = False:
            dict := {}:  // 初始化字典
```


中国知网 https://www.cnki.net

```
        if node.output_dimensions > 4:  // 输出维度大于 4
            dict[node.id] := node  // 保存这个 reshape 算子
            recursive_reduce(node, dict)  // 进入递归搜索降维算法
    return G
function recursive_reduce(node, dict):  // 递归搜索降维
    successors := get_successors(node)  // 获取当前算子的后置节点
    if length(successors) = 1:
        successor := successors[0]  // 获取后置节点
        if successor.type = 'Add':  // 若后置节点是 Add 算子
            input_other := get_other_input(successor, node)  // 获取算子其他输入
            if input_other.type ∈ ['Unsqueeze', 'Reshape']:
                dict[other_input.id] := other_input  // 保存该输入到字典中
            elif other_input.type = 'Add':  // 如果是 Add
                进行溯源，若全是 Unsqueeze 或 Reshape，则不可降维，return
                否则将这些溯源算子添加到字典中
                successor := get_successors(successor)  // 获取下一个节点
                if successor.type = 'Add':  // 后置节点是 Add
                    recursive_reduce(successor, dict) // 递归
                elif successor.type ∈ ['Reshape', 'Squeeze']:
                    跳出递归，尝试降维字典中的算子
    return
end
```

3.4.2 基于递增子序列合并的 Transpose 算子降维算法

一些在 Transformer 模型中，尤其是涉及多头自注意力机制（Multi-Head Attention）的部分，Reshape-Transpose-Reshape 是非常常见的算子组合。一般来说这些算子主要用于调整张量的形状以适应计算需求，并有效地实现并行化计算，但是对于 NPU 来说，并不支持 5 维以上的算子，因此需要尽可能将这些算子进行降维。这样做的理论基础是，Transpose 算子的参数中存在连续递增序列

如图 3-5(a)，Transformer 模型中 Transpose 算子的参数为(0, 1, 3, 2, 4, 5)，而图 3-5 (b)中 Transpose 算子参数为(0, 2, 3, 1, 4)，这些参数中均含有连续递增子序列，而如果将这些子序列所在维度进行合并再进行 Transpose 推理，结果是不变的，根据这一特性，提出基于递增子序列合并的 Transpose 算子降维算法。

中国知网 https://www.cnki.net

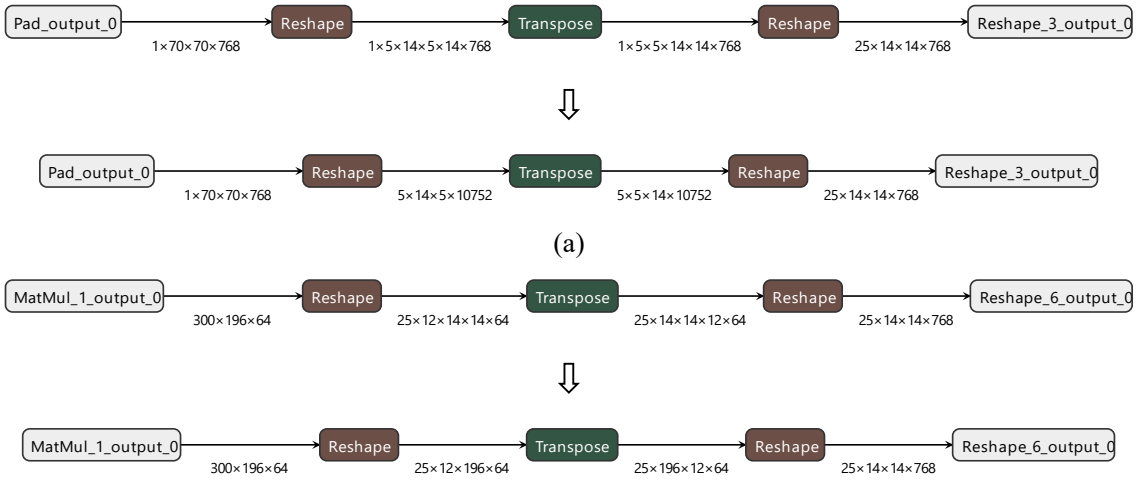


图 3-5 Transpose 降维

算法 3-2 是完整的基于递增子序列合并的 Transpose 算子降维算法，算法根据给定的 Reshape 的参数和它连接的 Transpose 的参数，合并 Transpose 参数中递增子序列的维度，并返回新的 Reshape 参数和和 Transpose 参数。算法主要流程如下：

（1）首先遍历 Transpose，将连续递增的维度归为同一组，并记录在 group 列表中。

（2）接着根据 group 列表计算出每个子区间的连乘结果，得到新的 Reshape 的参数。

（3）最后根据 group 列表，将列表元素按照大小递增排序得到位次，再使用位次替换原 group 中的对应的元素，替换的最终结果列表就是新的 Transpose 维度排列的参数。

算法 3-2：基于递增子序列合并的 Transpose 算子降维算法

输入：Reshape, Transpose ，是两个整数列表，代表 Reshape 和 Transpose 算子的参数

输出：ReshapeNew, TransposeNew ，是新的 Reshape 和 Transpose 算子的参数

```
function calculateNewShape(Reshape, Transpose):
    // 步骤 1: 遍历Transpose算子的参数，合并连续的维度
    groups := [ ]  // 存储合并后的维度组
    currentGroup := [transpose[0]] // 初始化当前组，包含第一个维度
    for i from 1 to length(transpose) − 1:
        if transpose[i] = transpose[i − 1] + 1:
            currentGroup.append(transpose[i])  // 如果递增，加入当前组
        else:
```


中国知网 https://www.cnki.net

10: $groups.append(currentGroup)$ // 否则，将当前组添加到$groups$
11: $currentGroup \coloneqq [transpose[i]]$ // 重置当前组为新维度
12: $groups.append(currentGroup)$ // 添加最后一个组到$groups$
13: 将$groups$按照每个元素（是一个列表）第一个元素从小到大排序
14: // 步骤 2: 计算新形状值
15: $Reshape_{New} \coloneqq [\,]$ // 空列表，存储新的形状
16: **for** $group \in groups$: //计算子数组的乘积作为$Reshape_{new}$参数
17: $Reshape_{New}.append(\prod_{x \in group} Reshape[x])$
18: $Transpose_{New} \coloneqq [\,]$ // 空列表，存储新的转置顺序
19: $k \coloneqq 0$ // 记录每个元素的位次
20: **for** i **from** 0 **to** $length(Transpose) - 1$:
21: **if** $i > 0$ **and** $Transpose[i] = Transpose[i-1] + 1$:
22: **continue** // 若当前元素为前一个元素+1，直接跳过
23: // 否则在$groups$中寻找当前元素所在的$group$的位次，即为新的替换值
24: ***for*** j ***from*** 0 ***to*** $length(groups) - 1$
25: ***if*** $\boldsymbol{Transpose}[i] \in \boldsymbol{groups}[j]$:
26: $Transpose_{New}.append(j)$
27: **break**
28: **return** $Reshape_{New}, Transpose_{New}$ // 返回新的$Reshape$和$Transpose$
29: **end**

3.5 算法性能测试与实验结果分析

为了验证降维算法对于 Transformer 模型的有效性，将 SAM_b 模型部署至 NPU 进行实验分析。本实验在 RK3588 EVB 开发板上，使用 RRT 开启 NPU 三核并行模式部署，而不使用 ORT。

表 3-3 显示了经过 Add 算子与 Transpose 降维算法优化以后推理时间排行前五的算子种类，所有的 Add 算子均可以使用 NPU 部署，从而消去了使用 CPU 部署的数据传输时间开销，此时 SAM 模型推理时间为 28.93s，总推理时间相比于优化前 136.244s 的下降 78.76%。

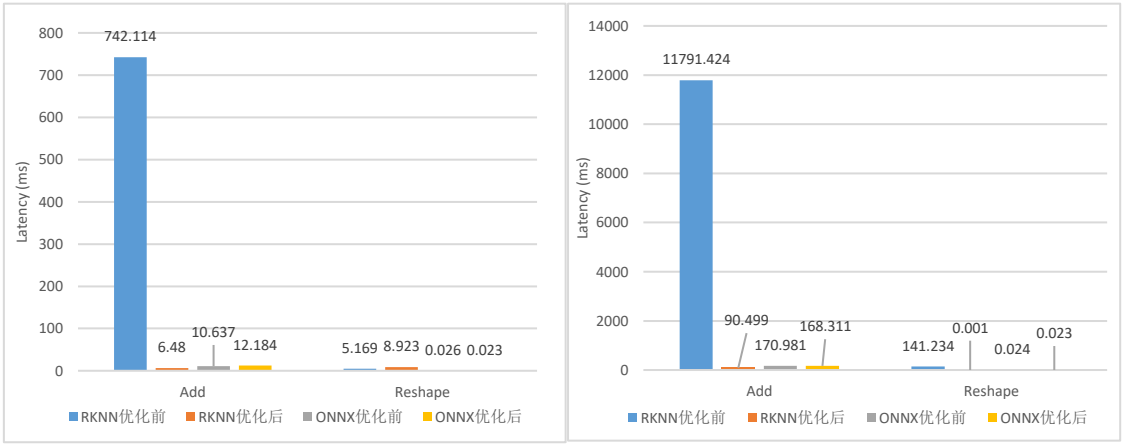
对部分 5 维或 6 维的 Transpose 和 Reshape 算子，优化后时间有所下降，但仍有大部分时间开销来源于 CPU，这可能是优化过程中的一个瓶颈，值得进一步研究。除此以外，有相当一部分数量的 exSoftmax13 算子使用 CPU 进行推理，这可能是因为 RRT 不支持 NPU 推理大规模的 exSoftmax13 算子。

中国知网 https://www.cnki.net

表 3-3　SAM_b 模型优化后各层推理时间

算子类型	数量	CPUTime(us)	NPUTime(us)	TotalTime(us)	TimeRatio(%)
Transpose	148	17077157	1723849	18801006	64.98%
exSoftmax13	12	605640	4488126	5093766	17.61%
Reshape	152	598100	918512	1516612	5.24%
exMatMul	48	0	1233064	1233064	4.26%
Add	33	0	833569	833569	2.88%

图 3-6 为 SAM_b 模型优化前后部分维度 Add、与 Reshape 算子的推理时间对比，首先是 RKNN 模型时间对比：规模为 300x14x14x14x14 的 Add 算子优化前需要 742.114ms，优化后只需要 6.48ms，总时间减少 99.126%，但相比与 Add 算子，Reshape 算子优化前为 5.169ms，优化后为 8.923ms，耗时却增加了；而规模 12x64x64x64x64 的 Add 算子优化前需要 11791.424ms，优化后只需要 90.499ms，总时间减少 99.232%。Reshape 耗时从 141.234ms 降至到 0.001ms，几乎可以忽略不计。这说明 RRT 内部实现 Reshape 的机理较为复杂不易掌握。而对于 ONNX 模型，两种算子的推理时间几乎没有变化。这说明 ORT 对于形状的维度并不敏感。并且 Reshape 在 ORT 上推理时间均为微秒级别，几乎可以忽略不计。Reshape 在 ORT 上推理时间远超 RRT，这可能是因为 RRT 推理时只能使用 1MB 大小的 SRAM 所致。



(a)300x14x14x14x14　　(b)12x64x64x64x64

图 3-6 优化前后 Transformer 算子推理时间对比

图 3-7 为 SAM 模型优化前后部分维度 Transpose 算子的推理时间对比，对 RKNN 模型来说，其中规模为 1x5x14x5x14x768 优化前需要 17.101ms。优化后只需要 10.615ms，耗时减少 37.92%；而规模为 25x12x14x14x64 优化前需要 18.23ms，

中国知网　https://www.cnki.net

优化后只需要 5.877ms，耗时减少 67.76%。对 ORT 来说，1x5x14x5x14x768 规模优化前后分别是 1.887ms 和 1.861ms，几乎没有变化，25x12x14x14x64 规模优化前 5.749ms，优化后为 2.601ms，耗时减少 54.75%。因此这种方法也适合 ORT。

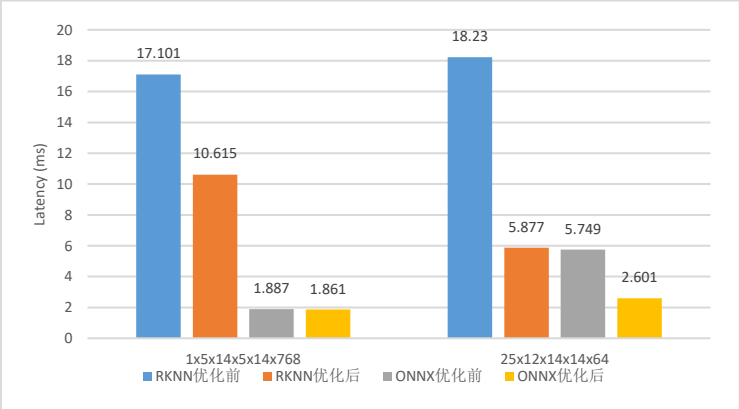


图 3-7 Transpose 优化前后推理时间对比

因此这种降维算法对 Element-Wise 算子、Reshape 算子和 Transpose 算子都十分有效，且对 Element-Wise 算子提速能达到 99%以上，对于 Transpose 算子在 RRT 框架中提速最高能达到 67%，而在 ORT 框架中也能得到一定的提速。然而相对于 ORT 来说，Transpose 和 Reshape 算子使用 RRT 性能不如 ORT。因此本文将在下一节提出模型分区算法，旨在将适用于 CPU 运行的算子部署至 CPU，适用于 NPU 运行的算子部署至 NPU，以此进一步提升模型推理速度。

3.6 本章小结

本章聚焦于 NPU 在深度学习推理加速中存在的优化不完善和高维算子支持度较低的问题，提出了两大类计算图优化算法，并通过实验验证了这些算法的有效性和可行性。本章开始时明确了当前 NPU 技术在实际 Transfomer 模型部署过程中遇到的挑战，即优化机制的不足以及对高维算子的支持有限，这些问题制约了 NPU 在深度学习推理中的性能表现。针对高维算子的问题，设计了基于字典索引搜索的 Add 算子降维算法和基于递增子序列合并的 Transpose 算子降维算法两种降维算法，旨在避免因维度过高而导致被迫使用 CPU 来处理的情况。本章提出的两种算法为解决 NPU 在深度学习推理中的优化难题提供了有效的路径，同时也为第下一章模型分区研究奠定了基础。

中国知网 https://www.cnki.net

第4章 基于时延预测的模型分区与调度算法研究

4.1 引言

在 RK3588 上运行深度学习模型时，由于存在两种不同的推理框架使用不同计算资源，而这两种推理框架对于一些算子推理性能具有显著的差异性，因此如何高效地分配计算资源成为一个亟待解决的问题。

如图 4-1 为本章研究内容总体框架图，本章将介绍一种基于时延预测的模型分区算法，该算法能够根据模型各部分的计算成本离线地调整其执行位置，即决定哪些部分由 CPU 负责，哪些部分交给 NPU 处理。其次，针对矩阵乘法算子在转换为 RKNN 格式后出现低效的 Transpose 算子的问题，在 ONNX 模型中的 MatMul 算子前后引入冗余的 Transpose 算子组合。此外，还设计了一种基于 EDF 的多模型调度策略，旨在高效地利用计算资源的同时使任务平均响应时间减少。本章的实验部分将进一步验证所提方法的有效性。

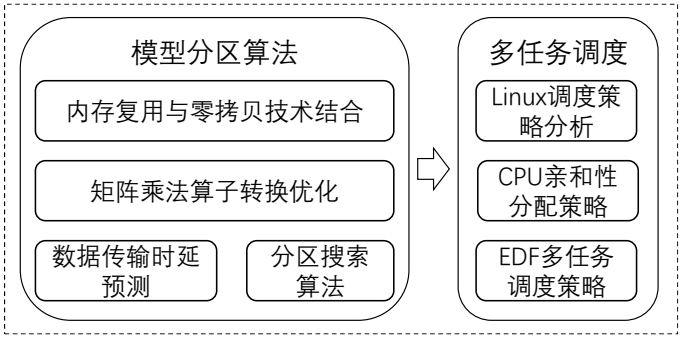


图 4-1 研究内容总体框架

4.2 面向单一模型的基于时延预测的模型分区算法

在移动设备中采用异构处理器（如 GPU 或 NPU）与 CPU 协同工作是常见的做法。选择最适合每个算子的处理器已被证明是一个 NP-hard 问题，因此现有方法通常依赖于机器学习来获取近似解，例如 CoDL 方案[31]。这种方法面临两大挑战：

数据集获取与训练成本：为了获得足够的数据集以适应不同的硬件特性，需要在移动端部署多种模型进行测试。由于不同硬件平台的推理框架对各个算子有不同的计算方式、融合规则和优化规则，完全拟合这些特性需要大量的数据集，这不仅增加了数据收集的复杂性，也延长了训练所需的时间。

数据传输延迟：RK3588 独特的硬件架构要求在 NPU 和 CPU 之间的数据交换

中国知网 https://www.cnki.net

时进行格式转换，而现有的模型分区方法往往忽略了这一因素对数据传输延迟的影响。

根据第 2 章的分析，当前 NPU 上的推理框架在处理 Reshape 和 Transpose 操作时表现不如 CPU，但在执行 MatMul 和 Conv 算子时速度远超 CPU。因此，理论上可以通过分配不同类型的算子到最合适的计算资源上来加速推理过程。然而，在 CPU 和 NPU 之间切换推理框架会导致显著的数据传输延迟，这也是目前基于算子或层级粒度模型分区算法在 RK3588 平台上性能不佳的原因之一。

为了解决这些问题，本小节提出了一种结合 RKNN 零拷贝与内存复用 API 的方法，旨在将数据排列格式转换的工作从 NPU 转移到 CPU 上，并消除了 NPU 内存区域至非 NPU 内存区域的数据拷贝步骤，并通过 ARM neon 指令集改进数据排列格式转换，从而减少数据传输时间开销。此外，设计并实现了一个基于随机森林回归的数据传输延迟预测模块，并研究出一种基于此预测的模型分区算法。

4.2.1 RKNN 零拷贝与内存复用结合

根据第 2 章的分析，当前调用神经处理单元（NPU）进行推理时存在显著的数据传输时间开销。这是由于 RRT 默认采用通用 API 来处理数据流程，导致在执行推理之前，必须先将输入特征图复制到 NPU 运行时内存中，然后再从该区域加载到 NPU 核心缓冲区，从而增加了额外的时间消耗。当数据规模达到 1x128x200x200 时，这一过程可能使数据传输时间高达 100 毫秒。即使对于较小的数据集，数据传输时间也往往与推理时间相当。

此外，部署图像分割模型 SAM_b 需要大约 4GB 的内存空间，而 RKNPU 使用 32 位地址设计，只能提供最多 4GB 的内存支持。这可能导致 SAM_b 模型无法成功部署，并且在多模型环境中，SAM_b 也无法与其他模型并发运行。

为了改善这种情况，降低模型推理的整体耗时，建议采用零拷贝 API 以减少不必要的数据复制步骤。同时，针对单个 NPU 核心仅能访问 4GB 内存的问题，当模型的中间状态内存需求超过此限制时，应该引入内存复用 API。通过结合零拷贝和内存复用技术，可以构建一个更高效的推理系统，有效解决上述挑战并提升性能。

综上所述，优化后的策略是整合零拷贝技术和内存复用机制，以减少数据传输延迟，并确保大型或多个模型可以在有限的内存资源下顺利执行。

图 4-2 展示了如何将两种 API 结合使用：创建初始化 RKNN 上下文之后，分别使用该上下文为模型权重、模型中间张量、模型输入与输出创建内存，再进行

中国知网 https://www.cnki.net

推理。对于多个模型，只需将其权重、中间张量、模型输入与输出的内存指定为第一个模型对应的内存区域接着顺序执行。

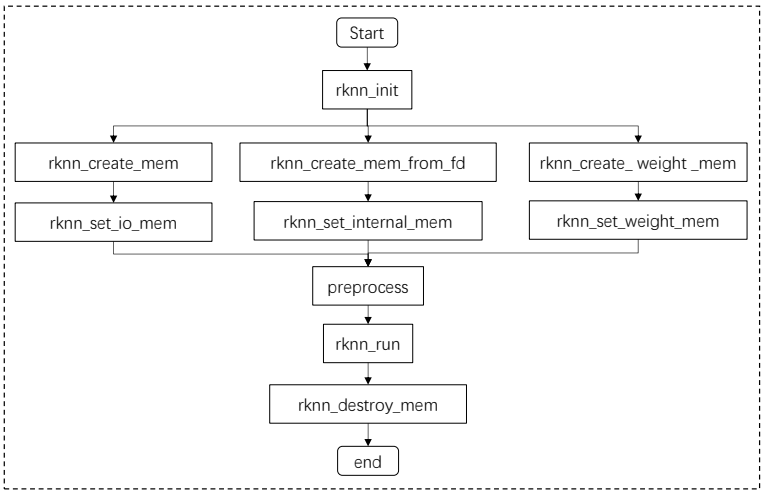


图 4-2 RKNN 零拷贝与内存复用结合

（2）RKNPU 数据排列格式转换

为了结合零拷贝技术使用不同的硬件进行协同计算，需要遵循不同的推理框架（即 ONNX 与 RKNN）下的数据排列格式进行相应的转换。目前 RKNN 的数据排列格式主要有四种：NHWC、NCHW、NC1HWC2、UNDEFINE。其中 NHWC 和 NHWC 数据排布为深度学习常见数据排列方式，NC1HWC2 是 RKNPU 硬件专用的数据格式，如图 4-3 所示， 数字 0 代表一笔数据，即一次存放 C2 个数据，其中 C2 是由平台决定的，RK3588 平台下的 C2 的规则约束为：int8 类型为 16，float 类型为 8，而$C_1 = \lceil C/C_2 \rceil$。NC1HWC2 数据存放的顺序与图中数值增长的顺序一致，先存放 0-15 的数据，再存放 16-31 的数据，以此类推。

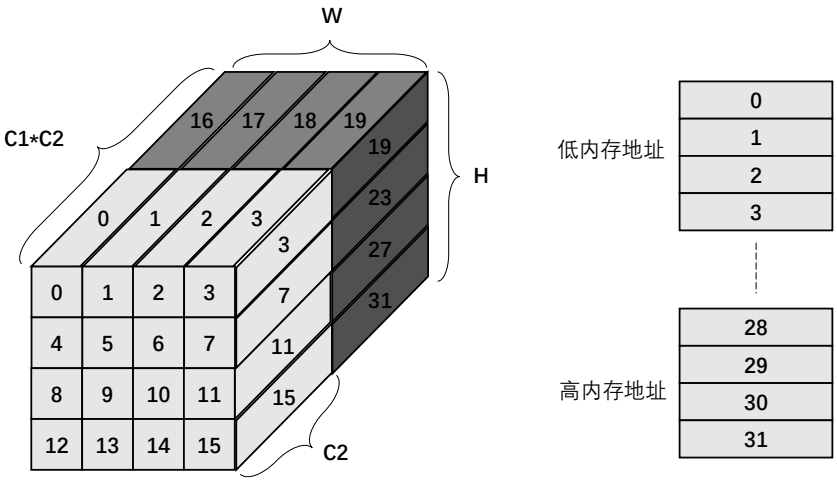
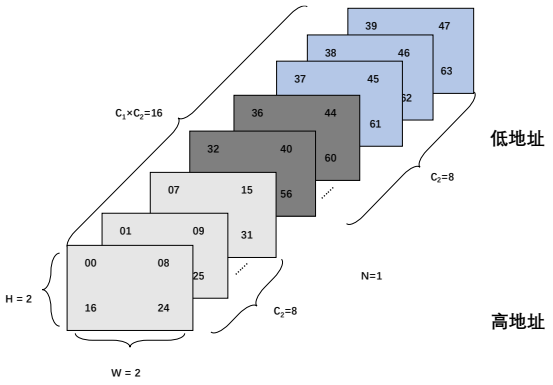


图 4-3 RKNPU NC1HWC2 数据排布与存储

与 RRT 不同，ORT 使用 NCHW 格式，因此这两种推理框架之间存在数据格

中国知网 https://www.cnki.net

式转换的问题，如果不使用零拷贝技术，则格式转换由 RKNN 模型推理结束后由 NPU 自动执行，但是由于使用了零拷贝，所有需要手动进行格式的转换。然而这会带来数据不对齐的问题，因此这里介绍一种 NC1HWC2 数据排列与 NCHW 数据排列互相转换的算法。如图 4-4(a)，ORT 使用特征图使用 float16 类型，维度为[N, C, H, W] = [1, 13, 2, 2]，而根据对齐要求可知将其格式转换为 NC1HWC2 后，数据排列如图 4-4(b)为：低地址 0~31，高地址 32~63，其中地址 37~63 位为额外对齐的无效数据。移除无效数据转成 NCHW 格式后即[1, 13, 2, 2]的数据，在内存中的排布如图 4-5。



0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

(a) (b)

图 4-4 NC1HWC2 数据排布展开

低地址

0	1	2	3	4	5	6	7	32	33	34	35	36
8	9	10	11	12	13	14	15	40	41	42	43	44
16	17	18	19	20	21	22	23	48	49	50	51	52
24	25	26	27	28	29	30	31	56	57	58	59	60

高地址

图 4-5 NCHW 数据排布

算法 4-1 是完整的 NC1HWC2 转换为 NCHW 的算法，当 RKNN 模型的输出需要转换为 ONNX 模型的输入时使用该算法。

算法 4-1：NC1HWC2 转换为 NCHW 算法

输入：$src, dst, dims, channel, height, width$，其中$src$为 NC1HWC2 排布的数据的地址指针，$dst$为 NCHW 排布的数据的地址指针，$dims$为 NC1HWC2 排布的$shape$、$channel$、$height$、$width$分别是 NCHW 的 C、H、W

1： **function** $NC1HWC2_to_NCHW(src, dst, dims, channel, height, width)$:

2： // 初始化维度信息

3： $batch, C_1, hw_{src}, C2 \coloneqq dims[0], dims[1], dims[2] \times dims[3], dims[4]$

4： $hw_{dst} \coloneqq height \times width$ // 目标张量的 size

中国知网 https://www.cnki.net

```
// 遍历每个批次
for i from 0 to batch − 1:
  // 更新src和dst指针到当前批次开始的位置
  src_batch ≔ src + i × C1 × hw_src × C2
  dst_batch ≔ dst + i × channel × hw_dst
  // 遍历每个通道
  for c from 0 to channel − 1:
    plane ≔ c ÷ C2 // 确定当前通道在C1维度上的位置
    offset ≔ c mod C2 // 确定当前通道在C2维度上的偏移
    // 根据plane计算源张量中对应的起始位置
    src_c ≔ src_batch + plane × hw_src × C2
    // 遍历图像的每一个像素点
    for cur_h from 0 to height − 1:
      for cur_w from 0 to width − 1:
        cur_hw ≔ cur_h × width + cur_w // 计算当前位置的线性索引
        // 将数据从 src 复制到 dst，进行格式转换
        dst_batch[c × hw_dst + cur_hw] ≔ src_c[cur_hw × C2 + offset]
end
```

4.2.2 基于冗余算子组合的纯净矩阵乘法算子转换算法

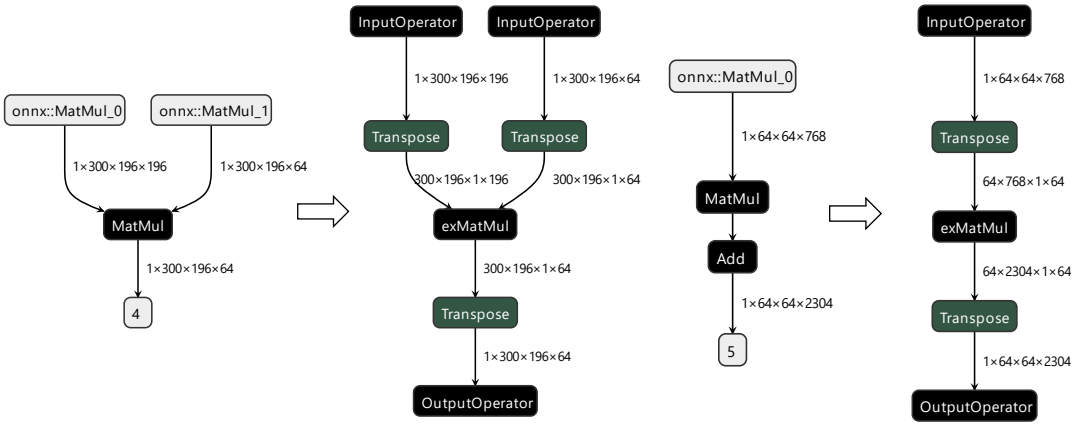
Transformer 模型中存在大量的 Transpose 算子与 Reshape 算子，同时还存在一定数量的 softmax 算子，原始的 Transformer 类大模型中，这些算子往往具有张量尺寸巨大的特点，如视觉基座大模型 Segment Anything 中大量 Transpose 算子的输入形状为[N=1，C=12, H=4096, W=4096]，这样的算子 RRT 推理框架有概率会将其从 NPU 核心的缓冲区转移到内存中，再调用使用 CPU 的后端推理引擎执行算子，但该引擎只使用单核 CPU 执行推理，造成推理效率低下。同时 RRT 是闭源的推理框架，这意味着开发人员只有遵循厂商提供的 RKNN API 进行开发，遗憾的是 RKNN API 并不支持修改 NPU 的推理过程。

在第 2 章的实验分析中了解到，在规模较大的时候，使用 CPU 进行 Reshape 与 Transpose 算子的推理速度远远快于 NPU，而大规模的 MatMul 矩阵乘法算子使用 NPU 计算速度快于 CPU，因此提出使用 RRT 与 ORT 相结合的部署 Transformer 类模型的策略，该策略为：一开始整个 Transformer 的 ONNX 模型使用 ORT 部署在 CPU 上，尝试将 MatMul 算子从 ONNX 模型中剥离出来转换为 RKNN 模型部

中国知网 https://www.cnki.net

署至 NPU，由于 NPU 对矩阵乘法算子的高支持度，达到模型整体加速的目的。

然而，在使用含有 MatMul 算子的 ONNX 模型转换 RKNN 模型的过程中，RKNN-Toolkit 会将 MatMul 算子进行修改以适配 NPU 运算单元，如图 4-6(a)，对于两个输入参数都为特征图的 MatMul 算子，RKNN-Toolkit 会将 MatMul 算子修改为 Transpose- exMatMul-Transpose 的形式；而如图 4-6(b)，对于 MatMul 中一个输入为初始化参数时，RKNN-Toolkit 会将 MatMul 算子修改为 Transpose-exMatMul-Transpose 的形式。然而，NPU 对于 Transpose 的支持度较低，不能使用多核并行推理，甚至在达到一定数据规模时，NPU 不仅会将 Transpose 算子迁移到 CPU 运行，还使用单核进行推理，这使得总体推理时间反而不如 CPU。因此提出面向基于冗余算子组合的纯净矩阵乘法算子转换方法。



(a) (b)

图 4-6 MatMul 算子优化前转换结果

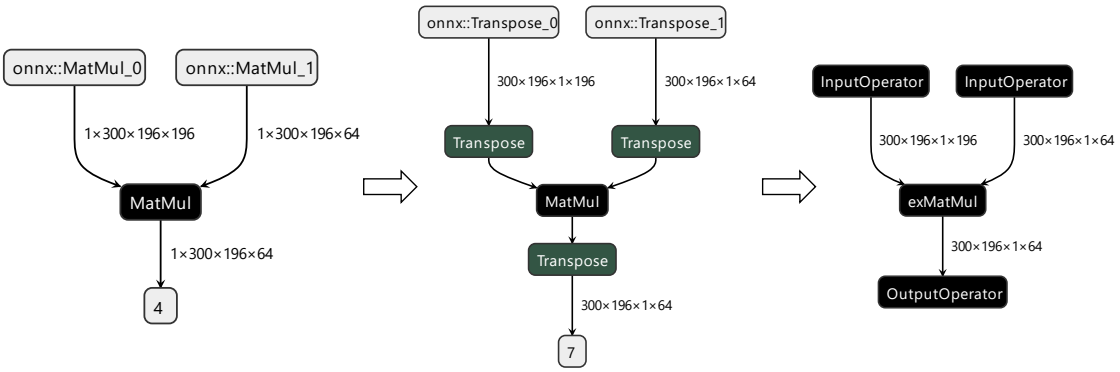
（1）冗余算子组合定义及其应用

冗余算子的定义是指的是在计算流程中没有贡献或其输出永远不会被使用的算子。冗余算子组合的图恒等变换则是指在计算流程或神经网络架构中，存在一组算子，它们的组合效果相当于一个恒等变换，通常可以表示为 f（X）=X，因此这些算子是不必要的，如连续的同一个激活函数算子组合、连续的归一化算子组合、连续的 Reshape 算子组合等。一般来说由于这些算子没有提供额外的价值，因此被认为是冗余的，然而，如果利用这些算子的特性，可以达到将一些 RKNN 模型中的算子提前拆分出来的目的。

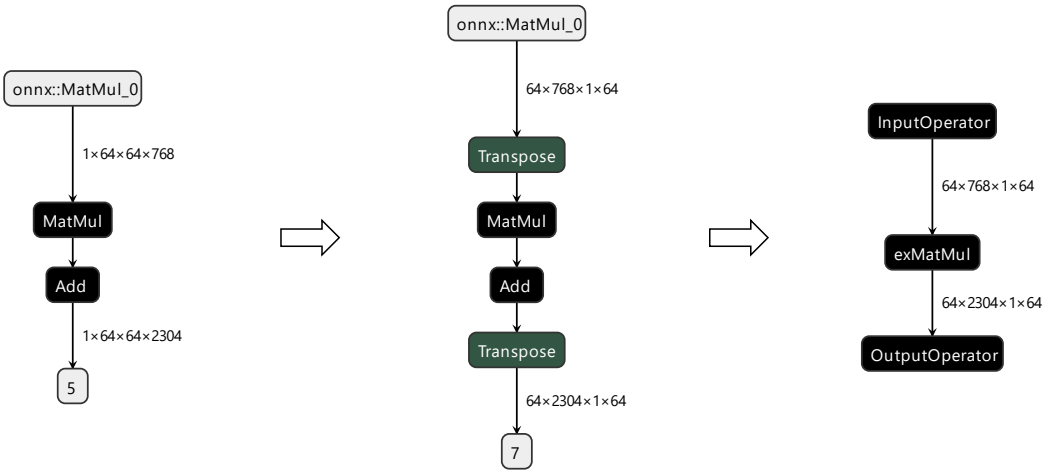
图 4-7，在原 ONNX 模型中的 MatMul 前后添加恒等变换冗余 Transpose 组合而使计算结果不发生变化，这样的好处是可以将组合中的其中一个 Transpose 算子拆分出来可以使用 CPU 运行，而另一个 Transpose 与 MatMul 算子绑定一起进入 RKNN 格式转换环节，这时 RKNN-Toolkit 会首先在 MatMul 前后添加 Transpose

中国知网 https://www.cnki.net

算子，接着发现冗余 Transpose 算子组合，将其从模型中移除进而单纯保留 exMatMul 算子，这样就可以将 MatMul 算子从 ONNX 模型中拆分出来部署到 NPU 上运行。



(a)



(b)

图 4-7 MatMul->exMatMul 模型简化

（2）双输入特征图的矩阵乘法算子转换方法

具有双输入特征图张量的矩阵乘法需要两个输入矩阵 A 和 B。对于输入矩阵 A，原始维度为 （b, k, 1, n），目标是转换到 (1, b, n, k)。应用转置组合 $T_A = [T_{A1}, T_{A2}]$，其中：$T_{A1} = Transpose(1,3,0,2)$ 将 A 转换为中间形式，$T_{A2} = Transpose(2,0,3,1)$ 作为 T_{A1} 的逆运算，最终得到目标维度。

对于输入矩阵 B：原始维度为 [b, k, 1, m]，无需改变，但为了保持一致性，同样应用一个恒等变换组合 $T_B = [T_{B1}, T_{B2}]$，其参数分别为 $T_{B1} = Transpose(1,2,0,3)$ 和 $T_{B2} = Transpose(2,0,1,3)$。

对于输出矩阵 Y：经过 MatMul 后，维度应调整回 [b, n, 1, m]。为此，在 Y 后

中国知网 https://www.cnki.net

添加转置组合 $T_Y = [T_{Y1}, T_{Y2}]$，其参数分别为 $T_{Y1} = Transpose(1,2,0,3)$ 和 $T_{Y2} = Transpose(2,0,1,3)$。

（3）单输入特征图的矩阵乘法算子转换方法

同理，对于具有单输入特征图的全连接层即矩阵乘法算子来说，将会转换为Conv算子的形式，其需要的输入矩阵A维度为[b, k, 1, n]，输出矩阵Y为[b, n, 1, m]，因此在原模型中的 MatMul 算子前与输入矩阵 A 后添加恒等变换冗余算子组合：$T_A = [T_{A1}, T_{A2}]$，其中$T_{A1} = Transpose(1,3,0,2)$，$T_{A2} = Transpose(2,0,3,1)$；在输出矩阵 Y 后添加组合：$T_Y = [T_{Y1}, T_{Y2}]$，其中$T_{Y1} = Transpose(1,2,0,3)$，$T_{Y2} = Transpose(2,0,1,3)$。

算法 4-2 是完整的基于冗余算子组合的纯净矩阵乘法算子转换算法。其中$insert_{transpose}$方法用于修改计算图 G，并返回修改的结果。具体算法流程如下：

（1）首先创建 6 种 Transpose 算子，并使用字典的方式将每个算子和其相关联的输入输出信息保存下来。

（2）接着按照拓扑顺序遍历计算图 G 中所有算子，当遇到 MatMul 算子时，遍历算子的所有输入节点，对于每一个输入节点，需要判断该节点属于矩阵乘法中的 A 矩阵还是 B 矩阵。若是 A 矩阵，则在输入节点与当前算子之间插入插入一对 Transpose 算子组合T_1和T_2，否则插入的是T_3和T_4。

（3）遍历 MatMul 算子的所有输出节点，对于每一个输出节点，判断该节点是否为 Add 或者 Mul，若是则在后置节点之后插入T_5和T_6，否则在当前算子和输出节点之间插入T_5和T_6。

（4）往复遍历所有算子直到计算图中算子均已遍历完毕。

算法 4-2：基于冗余算子组合的纯净矩阵乘法算子转换算法

输入：$G = (V, E)$，G是原 ONNX 模型计算图，V是算子节点集合，E是算子之间依赖关系

1： **function** $insert_{transpose}()$:

$T_1 := Transpose(1,3,0,2), T_2 := Transpose(2,0,3,1), T_3 := Transpose(1,2,0,3)$

$T_4 := Transpose(2,0,1,3), T_5 := Transpose(1,2,0,3), T_6 := Transpose(2,0,1,3)$

$nodes := \{node | node := \{$ // 使用字典保存节点相关联的边的信息

2：

$name := w.name,$

$type := w.type,$

$inputs := \{(u,v) | v = w, (u,v) \in E\},$

$outputs := \{(u,v) | u = w, (u,v) \in E\}$

$\}, w \in V$

中国知网 https://www.cnki.net

```
    }
3:  for node ∈ nodes: // 遍历 G 中的所有节点
4:    if node.type = 'MatMul':  // 如果节点是 MatMul
5:    for edge ∈ node.inputs: // 遍历节点的所有输入边
6:      predecessor := edge.u // 获取前置节点
7:      // 在前置节点 predecessor 与当前节点 node 之间插入 Transpose 算子组合
8:      if edge = A:
9:        如果输入边是 A 类型，插入T1和T2
10:     else:
11:       如果边是 B，插入T3和T4
12:   for edge ∈ node.outputs: // 遍历节点的所有输出边
13:     successor := edge.v // 获取后置节点
14:     // 检查后置节点的类型
15:     if successor.type = 'Add' or successor.type = 'Mul':
16:       如果后置节点是 Add 或者 Mul，在后置节点之后插入T5和T6
17:     else:
18:       在当前节点 node 和后置节点successor之间插入T5和T6
19: end
```

4.2.3 基于支持向量回归的数据传输时延预测模块

由第 2 章的分析可知 NPU 与内存之间存在一定的数据传输时间，而零拷贝也只是将数据排列转换的步骤转移到 CPU 上执行，或者根据具体模型使用情况决定是否需要转换，因此数据传输时间并没有完全消除或达到忽略不计；虽然数据传输时间和数据规模密切相关，但并不是线性关系，因此需要选择合适的非线性回归模型对其进行预测以用于模型分区指导。

（1）非线性回归模型

当前表现良好的用于非线性回归预测的模型有随机森林（Random Forest，RF）、梯度提升回归树（Gradient Boosting Regression Trees，GBRT）、多层感知机（MLP，Multi-Layer Perceptron）、支持向量回归 (Support Vector Regression，SVR)等。

随机森林是一种集成学习方法，它通过构建多个决策树并将它们组合起来以提高预测准确性和控制过拟合。其基本原理是通过聚合多个决策树形成一个强学习模型。随机森林中的每棵树都是基于原始训练数据集的一个随机子样本构建的，即从原始数据集中有放回地随机抽取样本，用来训练每一棵决策树。一般来说随

中国知网 https://www.cnki.net

机森林由于大量的随机决策树的构建使得模型不易过拟合，可以处理大量特征和噪声数据，训练速度快，可以并行化。

梯度提升回归树也是通过决策树构建，与随机森林不同的地方在于 GBRT 通过迭代地添加新的决策树来最小化损失函数。每棵树专注于减少前一棵树的残差（即预测误差）。GBRT 具有精度高的特点，能够容易获取复杂的特征与数据对应的关系，也因此带来过拟合的可能。一般通过调整超参数如学习率和树的深度来控制过拟合，但是带来的缺点是训练时间较长，并且由于通过前一棵树的残差来调整决策树，因此不能并行化。同时，GBRT 的参数调优较为复杂，对异常值较为敏感。

多层感知机是一种神经网络模型，它包含输入层、输出层以及隐藏层，每个神经元通过激活函数进行非线性变换，这也使得 MLP 能够捕捉非常复杂的非线性关系。MLP 常用于图像与文本等数据的预测或分类，也可以通过深度学习框架进行高效的训练，但是缺点在于需要大量的数据，并容易过拟合。

支持向量回归是一种基于支持向量机（Support vector machine，SVM）的回归预测模型，原理是寻找一个超平面使得它到最近的数据点的距离最大化。SVR 对于高维空间十分有效，在小样本数据集上表现良好，并且可以通过选择不同的和函数来处理线性和非线性问题。然而它对参数的选择很敏感，也因为对大规模数据集的训练速度慢而不适合大量样本的情况。

具体选择哪种模型需要进一步的评判，以下介绍两种评价指标。

（2）模型指标选择

为了对数据传输时延进行预测，需要比较不同的模型的性能差异。为了选择最优模型，使用均方误差 (MSE)与决定系数 (R^2 Score)作为评判指标，以确保模型具有良好的泛化能力。MSE 是衡量预测值与真实值之间差异的一种方法，它计算的是每个样本预测值与真实值之差的平方的平均值，适合用来比较不同模型在同一数据集上的表现。MSE 的定义由公式(4-x)给出：

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{4-1}$$

其中 n 是样本的数量，y_i是第 i 个观测的真实值，$\hat{y}_i$是第 i 个观测的预测值。R^2 表示模型解释的数据变异性的比例，即有多少数据预测发生了偏差。因此 R^2 可以用来比较不同模型对同一数据集的拟合效果。R^2 的定义由公式(4-x)给出：

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \tag{4-2}$$

其中，$SS_{res} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$是残差平方和，$SS_{tot} = \sum_{i=1}^{n}(y_i - \bar{y})^2$是所有数据

中国知网 https://www.cnki.net

的总平方和，$\bar{y}$是所有观测值的平均值。

（3）模型评估选择

为了评判这 4 种回归模型的性能差异，创建多种不同数据规模（即$[N,C,H,W]$四个维度为变化的值）的 600 项 Relu 算子获取经由零拷贝改进后的数据传输时延作为数据集。使用特征图各个维度即$[N,C,H,W]$与数据规模即 size=$N \times C \times H \times W$作为特征进行训练，借助 GridSearchCV 工具，使用 MSE 与决定系数作为评价指标进行交叉验证，从中选取表现最优的作为最终的模型。

表 4-1 是 4 种模型的指标对比，包含训练时间、推理时间、MSE 与 R2 core。在训练时间表现上 MLP 的训练速度最快，其次是 SVR。随机森林和 GBRT 的训练时间较长，尤其是 GBRT，它是所有模型中最慢的。这可能是因为 GBRT 构建了大量的决策树，每棵树都试图纠正前一棵树的错误，因此训练过程复杂且耗时。在推理时间方面，GBRT 和 SVR 的表现最好，它们的推理速度非常快。MLP 也有较快的推理速度，但随机森林的推理时间相对较长。这是因为随机森林需要聚合多个决策树的结果，增加了推理的计算负担。从 MSE 来看，MLP 的表现最差，其误差远高于其他三种模型。而 SVR、随机森林和 GBRT 的 MSE 都非常低，其中 SVR 的 MSE 最低，说明它在这方面的表现最佳。所有模型的 R^2 都很高，特别是 SVR 几乎达到了完美拟合的程度。相比之下，MLP 的 R^2 较低，表明它在解释数据变异方面不如其他三个模型。综上所述，选择 SVR 作为时延预测模型是最佳选择。

表 4-1　回归模型性能比较

Model	训练时间(ms)	推理时间(ms)	MSE	R^2
随机森林回归()	38079.80	20.98	45.22	0.998
多层感知机(MLP)	712.45	3.77	6786.41	0.910
梯度提升回归树(GBRT)	40357.30	2.24	65.31	0.998
支持向量回归(SVR)	25501.03	2.51	37.71	0.999

4.2.4 基于时延预测的模型分区算法

一个计算图可以抽象为$G = (V,E)$，V 是算子节点集合，E 是边集，特点是 G 是有向无环图 DAG ，每个算子可以有多个输入边和多条输出边。同时也是算子输入输出关系集合。首先使用 Kahn 算法获取计算图中算子的 topo 序列 A = $[a_0, a_1, a_2, \ldots a_{|V|}]$，同时将节点名称与节点、节点的输入输出边用字典保存，将输入输出边、边的 source 算子节点集合和 target 算子节点集合用字典保存。A 中的元素保存了节点、节点序号、单个节点在 CPU 上和 NPU 上的推理时间。

由第 2 章分析可得知对于 Element-Wise 算子，NPU 推理速度略快于 CPU；对

中国知网 https://www.cnki.net

于矩阵乘法和卷积类算子，在计算规模较大时 NPU 推理速度远快于 CPU；而对于数据搬运类算子如 Reshape 与 Transpose，CPU 推理速度远快于 NPU。因此可以将适用于 CPU 运行的算子部署至 CPU，适用于 NPU 运行的算子部署至 NPU。基于此特点，提出以下模型分区算法：

（1）按照 topo 序列遍历 V，将 Conv、MatMul、和 MatMul-Add 算子组合的标记为使用 NPU 执行，将 Transpose、Reshape 标记为使用 CPU 执行；

（2）将这些被标记位置中相邻位置用同一类型处理器部署的算子或算子组合的下标在 A 中的标记出来作为同一子区间；

（3）以这些子区间作为起点，记下起点的推理硬件（即 CPU 或 NPU），从子区间边界分别向两端开始，搜索最佳分割点。具体来说，对于每一个被搜索的节点a_P，将其 topo 序列之前的算子节点与其之后的子区间的算子节点之间的所有边找出来作为待预测集合E_{a_P}，由数据传输时延预测模型读取这些边的维度，得到预测的数据传输时间。将数据传输时间求和得到t_{a_P}，保存子区间之间被搜索的节点得到的t_{a_P}，选择最小值对应的节点 P，即为最佳分割点。

（4）搜索直到下一个区间的边界结束，最终得到所有的各子区间之间的最佳分割点a_p与对应的输入与输出边集合E_{a_P}；

（5）碎片整理：此时可能会产生很多子区间碎片。子区间碎片是指在短暂的算子数量区间内有大量的交叉使用不同处理器的子区间，即$N = \{n_0, n_1, \dots, n_k\}$，其中$n_i$是第 i 个子区间，该子区间全部使用同一种类型的计算资源，即 CPU 或 NPU。对每一个子区间尝试部署到上一个子区间的相同处理器上，若此时推理时间小于原处理器上推理时间+数据传输时间，则尝试成功，将该子区间计算资源标记为新处理器。

（6）根据每一个E_{a_P}进行 ONNX 模型分区与 RKNN 模型格式的转换，得到模型集合$M = \{model_0, model_1, \dots model_{p-1}, model_p, \dots, model_m\}$，其中每一个元素是原模型的一个子模型，保存了在 CPU 或 NPU 上执行的标记。且满足$\forall 0 \le i < j \le m, model_j$在$model_i$执行结束之后执行。

如图 4-8 为分区后各模型的运行框架，将 N 个模型进行分区优化以后得到 m 个子模型。其中$Model_1$与$Model_2$是同一个 Model 的子模型，由于实现了内存复用，其权重与中间内存使用同一片内存区域$Area_{model_1}$，$Model_{m-1}$与$Model_m$是同一个 Model 的子模型。由于实现了零拷贝，所以$input_1$、$output_1$、$input_2$、$output_2$使用同一片内存区域$Area_{io_1}$。$output_1$是$input_1$输入$Model_1$后的推理结果，$Model_2$将使用它作为$input_2$输入。

中国知网 https://www.cnki.net

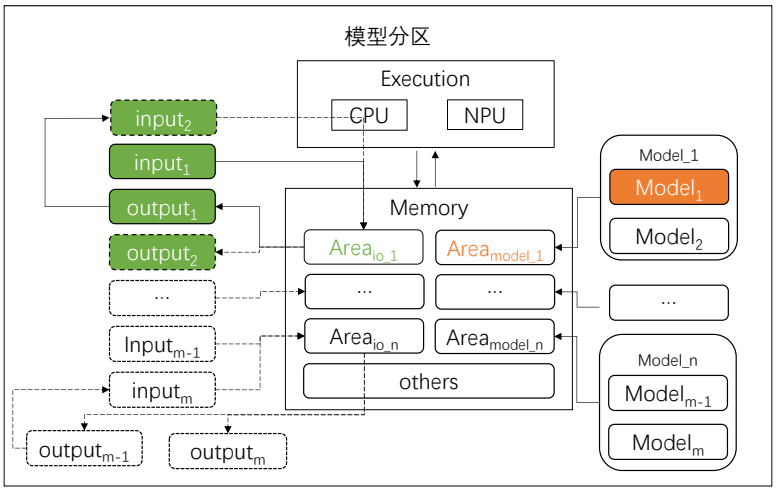


图 4-8 模型分区功能总体框架

4.3 面向多模型的最早截止时间优先调度策略

通过 4.2 模型分区技术可以将一个完整的 DNN 模型划分为多个子模型，然而现有的 Linux 系统调度策略在顺序执行这些子模型时存在一些不足之处，例如并未考虑模型之间的连续性，也并未考虑有的子模型使用 NPU 会频繁触发系统中断，导致进程切换的开销等。

为了解决以上问题，更高效地依照子模型之间的依赖关系执行这些模型，同时为了在多任务场景下能更高效地部署模型，本小节从当前 Linux 系统调度策略出发探究现有调度策略的不足，提出基于大小核差异的 CPU 亲和性策略与基于 EDF 的多模型调度策略。

4.3.1 Linux 系统调度策略

（1）移动端 Linux 内核调度策略

当前针对多进程场景，移动端通常使用 CFS、EAS、HMP 等调度策略。CFS 完全公平调度器（Completely Fair Scheduler，CFS）是一种为所有进程提供公平的 CPU 时间分配，并且能够处理各种类型的负载。针对于异构多核处理器，也常使用 HMP (Heterogeneous Multi-Processing)调度器，HMP 调度器可以将任务合理地分配给高性能的大核心或者低功耗的小核心，以此来平衡性能与功耗，EAS (Energy Aware Scheduling)是一种基于 CFS 的扩展，它能够更智能地分配任务到不同的 CPU 核心上，从而达到节能的目的。它考虑了每个核心的能量效率以及当前的工作负载情况来决定哪个核心最适合运行某个任务。尽管 EAS 和 HMP 能够有效地平衡性能与功耗，但它们在处理实时场景特别是多深度学习推理任务的场景下并不能

中国知网 https://www.cnki.net

有效地调度。

（2）实时任务调度策略

为了满足具有严格时间限制的任务能够及时获得 CPU 资源的紧迫性需求，当前 Linux 系统中主要提供了两种实时任务调度策略即 SCHED_FIFO（先来先服务）和 SCHED_RR（轮转法），它们都为进程赋予了静态优先级，数值越大优先级越高。这些策略允许实时进程比普通进程拥有更高的优先级，确保实时任务能够在必要时抢占 CPU 执行。

EDF（Earliest Deadline First，最早截止时间优先）算法是一种广泛应用于实时系统中的动态调度算法。它的核心思想是根据任务的截止时间来确定任务的优先级：截止时间越早的任务，其优先级越高。因此系统会选择具有最早截止时间的任务来执行。EDF 在单处理系统中性能优异，它能够在理论上保证所有可调度的任务集合都能得到正确的调度。并且 EDF 特别适合那些对时间敏感的应用程序，因为它能够确保关键任务按照它们的截止时间来被调度和执行。这类应用场景包括但不限于嵌入式系统、网络数据包调度、多媒体播放等。

（3）当前调度策略存在的问题

当前调度策略存在以下不足：一是由于子模型优先级分配不合理，不能有效抢占计算资源，从而被阻塞；二是当前 CPU 的调度策略未能充分考虑 NPU 的执行状态，可能导致在 CPU 资源被抢占的情况下出现长时间的 NPU 空闲，进而延长了模型总体的平均响应时间，三是 NPU 可能随时发出中断，对于同一个 CPU，频繁的 CPU 进程切换可能会导致性能损耗。

为了使分区后的模型能够更高效地执行，并考虑到移动端混合计算任务执行的独特需求，提出基于 CPU 亲和性绑定策略与基于 EDF 的任务优先级分配策略。

4.3.2 基于 EDF 的多模型调度策略

（1）CPU 亲和性分配策略

一般来说，深度学习任务的运行包括数据预处理、模型推理、推理结果后处理，其中预处理与后处理部分一般使用 CPU 执行，模型推理部分使用 NPU 或 CPU 执行，如果模型使用了模型分区算法优化，则模型推理部分将分为多个连续的具有依赖关系的子任务。因此一个深度学习任务可以抽象为一个四元组 $\tau_i = (C_i, T_i, D_i, L_i)$，$C_i$为 WCET（最坏执行时间），$T_i$为最小到达间隔时间，$D_i$为任务的相对截止时间，$L_i$为划分后的子任务数（模型数据前处理与后处理看作子任务）。τ_i是一个 RT 任务，它有一个受限的截止日期，即 $D_i \leq T_i$。由第 2 章的分析可知，

中国知网 https://www.cnki.net

NPU 可能会容易触发中断造成 CPU 上下文切换，而本节提出的调度策略为一个子任务要么在 CPU 上执行，要么在 NPU 上执行，所以将使用 NPU 推理的任务进程与其中一个 CPU 小核绑定，该小核在 ORT 推理时不参与执行。

（2）基于 EDF 的多模型调度策略

RK3588 上基于视觉的应用基本上都为实时任务，因此需要一个良好的实时调度系统。幸运的是 Linux 实现了基于 EDF 算法的 SCHED_DEADLINE 策略，它为每个任务设定了三个参数：周期(sched_period)、截止时间(sched_deadline)和运行时间(sched_runtime)，基于此，使用基于 EDF 的多模型调度策略。其中优先级基于 SCHED_DEADLINE 策略自动进行分配。其中周期的计算：一个模型的子任务的周期为整个模型的周期，截止时间的计算：可以在任务开始执行前记录当前时间戳，在任务结束时再次记录时间戳，并计算两者之差；运行时间的计算：运行时间指的是任务在一个周期内被允许占用 CPU 的时间总量。测量运行时间的方法之一是在任务开始执行时启动一个计数器，并在任务结束时停止计数器，然后读取计数值即为实际消耗的 CPU 时间。

具体的调度策略为：将每一个深度学习任务进行划分（模型分区和前后处理与推理过程划分）以后，将它们按照模型之间的依赖关系进行多线程或多进程顺序执行。具体执行时，创建一个进程池用于任务管理，将这些任务分配给进程池等待被调度，每一个进程只能执行一个模型。

4.4 算法性能测试与实验结果分析

4.4.1 模型分区实验与结果分析

为了验证算法的可行性，使用 Transfomer 类模型测试模型分区优化效果，并将分区后的子任务分配按照 4.3 节所提到的调度算法进行部署测试，与单独使用 ORT 推理框架或单独使用 RRT 推理进行对比。

采用 RK3588 EVB 开发板作为测试环境，针对 RRT，使用 NPU 三核并行模式；针对 ORT，开启最大线程（即 8 线程）部署；而针对模型分区测试，同时将 NPU 三核并行模式与 ORT 最大线程模式开启进行部署。

如图 4-9 为使用未采用降维算法的 SAM_b 与 SAM_L 视觉大模型和同样基于 Transfomer 的 superglue 模型在 RK3588 上用三种推理策略的结果。从图表中可以看出不同模型在不同框架下的推理时间对比：Superglue 的推理时间在 ORT 和 RRT 上都比较低，几乎可以忽略不计，这表明该模型在 RK3588 上运行非常高效，其使用模型分区算法以后相比于 RKNN 推理耗时下降 16.46%，相比于 ORT，其推理

中国知网 https://www.cnki.net

耗时降低约 17.60%。对于 SAM_b 和 SAM_l 模型来说，它们使用纯 RKNN 进行推理时耗时非常长，而使用纯 ORT 框架时，性能远好于 RKNN。使用模型分区性能最佳，相比于 RKNN，SAM_b 耗时下降 91.44%，SAM_l 耗时下降 79.13%。相比于 ORT，SAM_b 耗时下降 6.23%，SAM_l 耗时下降 3.65%。因此算法是有效的，但总的来说 NPU 对于 Transformer 类模型的支持度还不是很好。

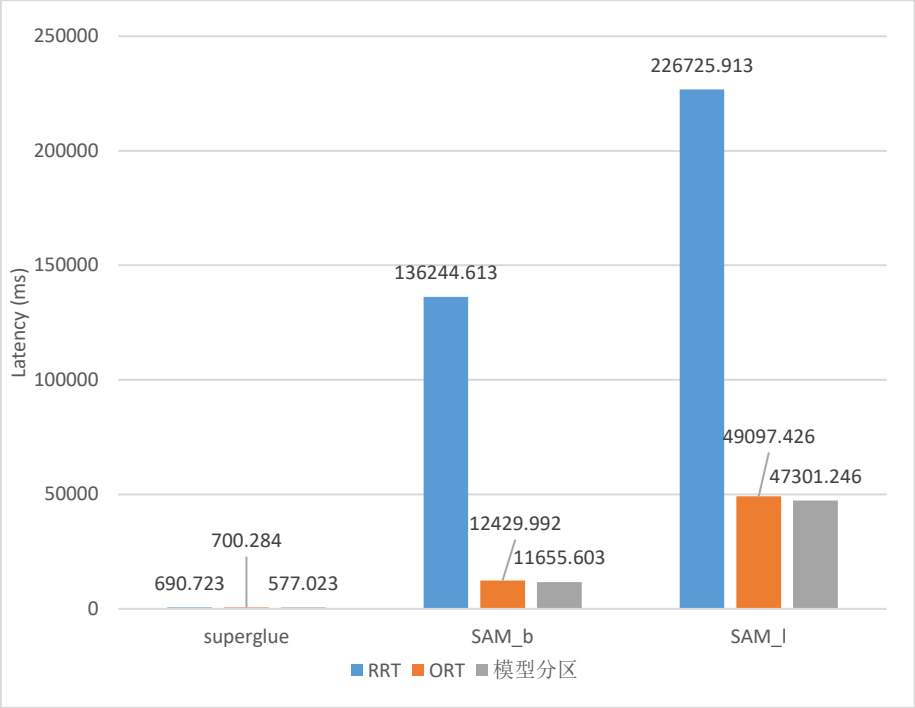


图 4-9 三种部署策略推理时间对比

4.4.2 多模型调度实验结果分析

为了验证多模型调度实验有效性，选择经典模型在 RK3588 上进行部署。在实际场景中，往往会同时使用到图像分割与目标检测的功能，基于 Transformer 的 SAM_b 模型是当下最佳的图像分割模型，并且 Yolov8n 模型也是性能优异的目标检测模型。因此实验选择 Yolov8n 与 SAM_b（未降维）进行模型分区后的子模型集进行多任务部署。部署分为两类策略，策略一为不使用 EDF 而使用默认的 FCFS 调度策略进行部署。策略二为使用 4.3 节的调度算法进行部署。实验中，同一个模型，一旦上一个任务结束马上发起下一次请求。

实验采用 RK3588 EVB 开发板作为测试环境，同时将 NPU 三核并行模式与 ORT 最大线程模式开启进行部署。

如图 4-10 是两种部署策略下各自运行 5 分钟的平均响应时间对比，结果显示，5 分钟之内，在策略一时任务的总平均响应时间较长，策略二任务平均响应时间较

中国知网 https://www.cnki.net

短，相较于策略一较少了 11.43%。对于 SAM_b 模型，其策略一中平均响应 21.13 次，策略二中平均响应时间 17.35 次，平均响应时间在策略一时较策略二短，为策略二的 91.57%；对于 Yolov8 情况则刚好相反，策略 1 中平均响应 1178.23 次，策略二中为 1336.84 次。

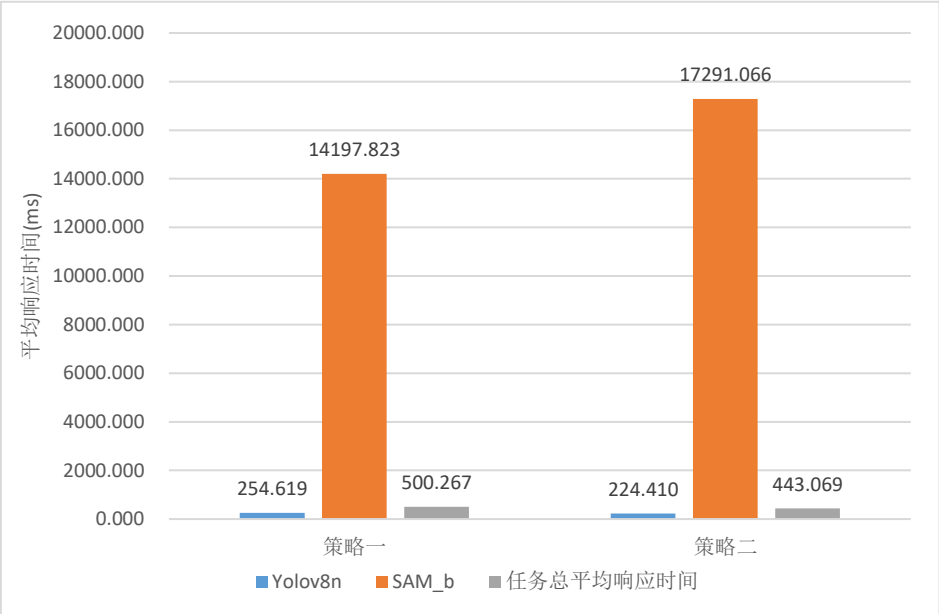


图 4-10 多模型调度实验结果

这是因为对于计算量大的任务（如 SAM_b），在 FCFS 调度下，它一旦开始执行就可以连续运行直到完成，减少了频繁上下文切换带来的开销。相比之下，在更复杂的调度算法（如 EDF 或其他优化调度）中，可能会因为试图公平分配资源或优化整体性能而引入额外的等待时间。这表明了 EDF 调度算法的可靠性。

4.5 本章小结

RK3588 处理器上优化深度学习模型推理性能时面临的一项重要挑战，是如何高效地分配计算资源给两种不同的推理框架，为解决这一问题，本章提出了一种基于时延预测的模型分区算法，旨在离线评估模型各部分的计算成本，并据此调整其执行位置。具体而言，该算法决定哪些计算任务由 CPU 承担，哪些则交由 NPU 处理，从而实现更合理的资源利用。针对矩阵乘法算子（MatMul）转换为 RKNN 格式后出现低效 Transpose 算子的问题，在 ONNX 模型中的 MatMul 算子前后添加了冗余的 Transpose 算子组合。这种方法有效地剥离了不必要的 Transpose 操作，使得单独的 MatMul 算子可以直接转换为 RKNN 格式并在 NPU 上高效运行。为了确保系统在多任务环境下的高效运作，本章还引入了 CPU 亲和性分配策略与基于 EDF（Earliest Deadline First）的多模型调度策略。策略根据任务的截止时间来动态

中国知网 https://www.cnki.net

调整它们的执行顺序，确保关键任务能够及时完成，同时减少了任务的平均响应时间，提高了系统的整体效率与用户体验。

实验结果表明，本章的模型分区算法最高可将 SAM_b 推理耗时减少 6.23%，将 SAM_l 推理耗时减少 3.65%。本章的多模型调度策略可将任务总体平均响应时间减少 11.43%。

中国知网 https://www.cnki.net

第5章 实验平台设计与实现

5.1 引言

为了实现高效的模型优化与部署，本章介绍了一个专为此目的而设计的平台，该平台结合了特定的硬件配置和定制化的软件系统，以满足不同类型深度学习模型的需求。通过这个平台，对当前主流的 Transformer 类模型、卷积神经网络（CNN）类模型进行了混合任务调度的测试。这些测试不仅评估了模型在优化前后的性能变化，还考察了它们在实际部署过程中可能遇到的各种挑战。最终的实验结果表明，实现了的推理速度提升。

5.2 实验平台搭建

5.2.1 平台整体框架

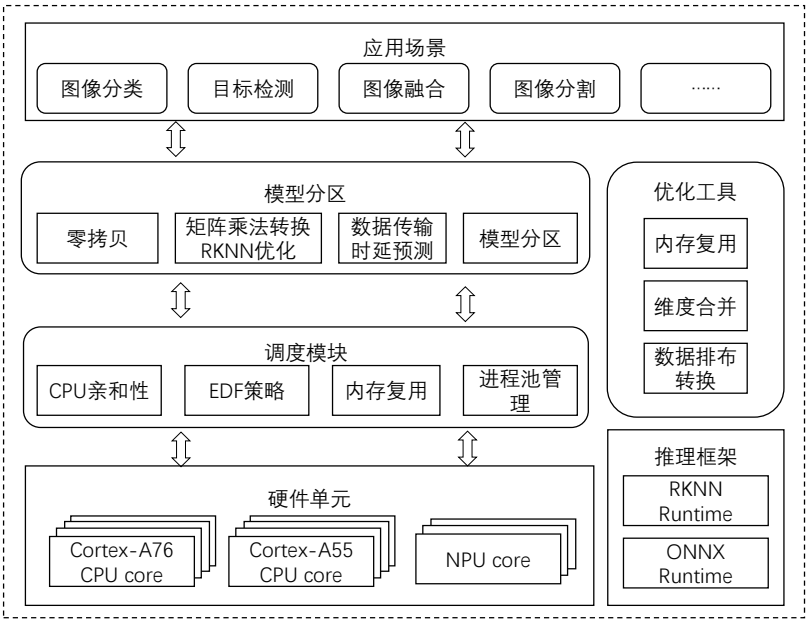


图 5-1 实验平台整体框架

如图 5-1 为实验平台整体框架，采用 RK3588 EVB 开发板作为开发平台（图 5-2），使用搭载 RK3588 处理器的智能头盔作为具体实验环境进行测试，该智能头盔搭载 8 核 CPU 与 3 核 NPU，测试时针对 CPU 使用 ORT 开启 8 线程部署，针对 NPU，使用 RRT 开启 NPU 三核并行模式部署。RK3588 处理器搭载的 CPU 与 NPU 型号与计算能力已在第 2 章详细介绍。

中国知网 https://www.cnki.net

这些应用需要使用 CPU 与其他硬件进行输入数据预处理如图像加载、图像伸缩变换、视频流读取等，也需要使用 CPU 进行具体的模型输出后处理如将推理结果进行过滤、将检测框画至视频读取界面，因此属于混合计算任务场景。

各个模型首先经过优化工具与模型分区模块结合优化，得到各模型子模型集，结合模型的前处理与后处理被划分为执行阶段，将其分别部署至已经配置好的工作线程池中等待被调度。后端使用 RKNN Runtime 与 ONNX Runtime 分别用于深度学习模型在 CPU 与 NPU 上的部署。



图 5-2 RK3588 EVB 开发板

5.2.2 应用软件设计与实现

（1）多进程调度程序设计与实现

基于 Python 多进程模块 multiprocessing 设计并实现了一个轻量级多任务可配置调度程序，为了使用 Linux 系统提供的 EDF 调度算法，使用 ctypes 与 c 语言封装 API 结合的方式使得 Python 调度程度可以直接调用该算法，结合 4.4 节的优先级分配策略，可以达到多任务实时调度目的。

程序总体流程如图 5-3，用户可以通过命令行输入想要运行的模型名称，该程序将根据模型名称匹配对应的 AI 任务，AI 任务将被放进任务队列中，由进程池进行选择，在运行过程中将根据第 4 章中优先级分配的方法进行优先级调控，确保进程能被更合理地调度。AI 任务将持续进行模型推理，直到推理次数达到用户设置的总次数或推理时间达到提前设置好的阈值，或者被用户发送 kill 命令杀死。为

中国知网 https://www.cnki.net

了能被再次更快速地响应，被停止的应用可能会被放入任务完成队列，用户若想再次启动该任务，任务选择器可以从任务完成队列选择先前被停止的任务进行启动。用户也可以通过下面介绍的上位机界面进行命令调控。

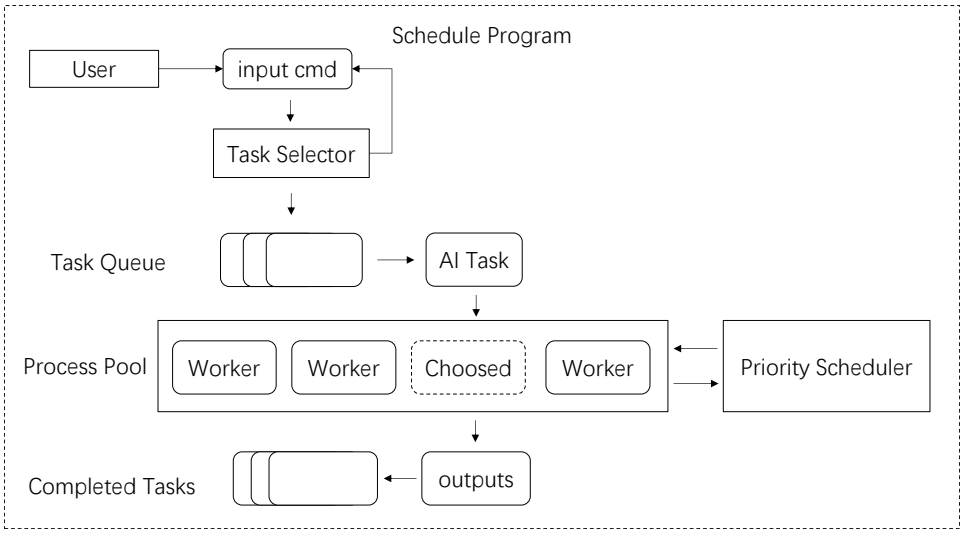


图 5-3 多进程调度程序架构

（2）上位机调控软件实现

为了更便捷地调控各 AI 应用，设计并开发了如图 5-4 所示的一个用于配置和管理深度学习模型的界面，主要功能包括选择 AI 应用、配置硬件模式、绑定 CPU 核心以及调控硬件频率。

在 AI 应用与平台各个硬件的运行情况调控方面，用户可以通过下拉菜单选择不同的 AI 应用进行配置和运行，并且可以分别设置与监控 NPU 模式、CPU 模式、DDR 模式和 GPU 模式。界面实现了绑定到特定 CPU 核心的功能，以优化资源分配和性能。用户可以设置不同的频率等级，包括 CPU 频率、DDR 频率和 GPU 频率，并通过多个“执行”按钮分别执行这些配置。

在模型优化与调度策略配置方面，界面将 AI 模型的优化步骤集成为一个“一键优化”按钮，用户只需要下拉菜单选择想要优化的模型，点击按钮即可生成对应优化模型并适配到应用中。并且界面提供了两类调度策略即系统默认调度策略与智能优先级分配调度策略。

最后，界面提供了一个“输入命令”框，允许用户直接输入命令进行执行。输出信息区域显示执行命令后的结果，方便用户查看。界面还监控 NPU 和 CPU 的负载情况，帮助用户了解当前硬件的使用状态。

本应用通过 Qt 语言编写，适用于 Windows 与 Linux 平台，并支持 PC 端通过 ADB shell 命令调控 RK3588 开发板，也可以直接部署到开发板上，为深度学习模

中国知网 https://www.cnki.net

型的部署和性能优化提供了全面的工具支持。



图 5-4 上位机调控软件

5.3 混合调度实验结果与分析

5.3.1 实验设计

本实验旨在评估深度学习应用采用本文中的方案优化后在实际应用场景中进行混合任务调度的综合性能。为此，构建了一个模拟环境，该环境能够反映现实世界中复杂多变的工作负载特性。通过这种方式，可以更准确地测试不同模型之间的协作效果以及它们共同处理多样化任务的能力。

（1）单一模型实验设计

对于单一模型的运行，对其同时使用降维算法和模型分区算法进行优化后部署，与未优化的模型部署对比。

（2）多模型实验设计

多模型场景下，首先对应用分别使用本文研究的优化技术进行优化，接着在 RK3588 上分别使用两种调度策略运行运行混合四种 AI 应用各 5~20 次，这里 SAM 模型已经使用模型分区算法进行优化。这 2 种策略将在实验结果与分析一节详细说明。

中国知网 https://www.cnki.net

5.3.2 模型选择

实验专注于评估和验证模型分区与混合任务调度策略的性能表现，通过部署一系列先进的计算机视觉应用来实现这一目标。如表 5-1，这些应用包括基于 ResNet50 的图像分类（基于输入给出类别以及置信度排序）、基于 Transformer 架构的 SuperGlue 图像配置、Segment Anything 图像分割、基于 Yolov8 的目标检测以及基于 SeAFusion 的图像融合，如图 5-5 是它们的实现效果。

表 5-1　头盔应用列表

深度学习应用	应用类别	模型类别	主要算子
ResNet50	图像分类	CNN	Convolution，Add
SuperGlue	图像配准	Transformer	MatMul，Transpose，Reshape
SAM_b	图像分割	Transformer	MatMul，Transpose，Reshape
Yolov8	目标检测	CNN	Convolution，Add
SeAFusion	图像融合	CNN	Convolution，Add



图 5-5 深度学习模型应用

选择这些深度学习模型不仅是因为它们在各自领域具有先进性和广泛应用，更重要的是，它们对计算资源的需求各不相同，这是因为 Segment Anything 模型是以自注意力机制为主的 Transfomer 类模型，意味着其中不含有卷积算子，而含有大量大尺寸的 MatMul、Reshape 与 Transpose 等算子，其中 Reshape 与 Transpose 算子在数据规模达到一定程度时 CPU 的执行效率远大于 NPU，而 MatMul 算子数据规模达到一定程度时情况则相反。而 Yolov8、SeFusion 与 ResNet50 是以卷积计算为主的模型，而 NPU 对于卷积算子的支持度与执行效率又高于 CPU，因此对这些模型进行的测试可以涵盖几乎所有的移动端混合计算任务的场景。

中国知网 https://www.cnki.net

5.3.3 实验指标

合理的实验指标能更好地反映实验的结果，因此选择合适的指标也尤为重要。下面依次介绍本小节所需的指标即以及计算表达式。

（1）CPU 与 NPU 平均利用率

CPU 平均利用率和 NPU 平均利用率是一个衡量 CPU 或 NPU 在一段时间内工作负荷的重要指标。利用率为 CPU 或 NPU 实际用于执行计算任务的时间相对于总的可用时间的比例。高利用率通常意味着更高效的资源使用，但也可能指示瓶颈或过载的问题。

$$Utilization_{average} = \frac{T_{active}}{T_{total}} \times 100\% \quad (5\text{-}1)$$

其中T_{total}是需要评估的整个时间段，T_{active}是 NPU 或 CPU 在T_{total}时间段内实际上正在执行计算任务的时间，而 RK3588 处理器具有 8 个 CPU 核心与 3 个 NPU 核心，因此实验中实际上使用的 NPU 利用率和 CPU 利用率为各核平均利用率的平均值，因此使用公式（5-2）与公式（5-3）作为实验指标。

$$Utilization_{average_{CPU}} = \frac{1}{8}\sum_{i=1}^{8} \frac{T_{active_{cpu_i}}}{T_{total_{cpu_i}}} \times 100\% \quad (5\text{-}2)$$

$$Utilization_{average_{NPU}} = \frac{1}{3}\sum_{i=1}^{3} \frac{T_{active_{npu_i}}}{T_{total_{npu_i}}} \times 100\% \quad (5\text{-}3)$$

（2）平均响应时间

平均响应时间是衡量系统性能的一个关键指标，它反映了从请求发出到接收到响应的平均所需时间。这个指标对于理解系统的响应速度和用户体验非常重要，尤其是在实时处理、网络服务或用户交互频繁的应用中，例如 RK3588 处理器搭载的头盔场景就非常适用于这个指标。平均响应时间的计算方式由公式（5-4）给出。

$$Response_{average} = \frac{1}{\mathrm{n}}\sum_{i=1}^{n} T_i \quad (5\text{-}4)$$

其中T_i是第 i 个请求的响应时间。n 是在给定时间段内所有被测量的请求数量。对于每一个请求，响应时间$T_i = T_{i_{end}} - T_{i_{start}}$，即该请求到达系统的时间（开始时间）以及系统完成处理并返回响应的时间（结束时间）。

5.3.4 实验结果与分析

（1）单一 Transformer 模型实验结果与分析

对实际应用场景中涉及到的 5 种模型采用降维优化与模型分区优化后，与未优化的模型进行对比，得到单一模型的优化实验结果。图 5-6 为其中两种以

中国知网 https://www.cnki.net

Transformer 结构为基础的图像分割和图像配置模型的对比结果。由图可知，对于 Transformer 模型来说，优化前后推理时间有了明显下降。相对于原先的最佳推理框架的推理时间结果，相比于 ORT 推理，SuperGlue 使用降维与模型分区后推理时间下降了 21.8%；相比于 ORT 推理，SAM_b 的推理时间下降了 11.37%。

这个结果说明模型分区与高维算子降维可以同时作用于 Transformre 类模型，并且为模型推理时间带来一定程度的降低。这是因为 Transformer 模型使用降维算法以后，有的被降维的算子可以在模型分区的决策中不与前后算子分区，从而以少量推理时间的代价换来数据传输时间的大幅降低，总推理时间得到进一步降低。

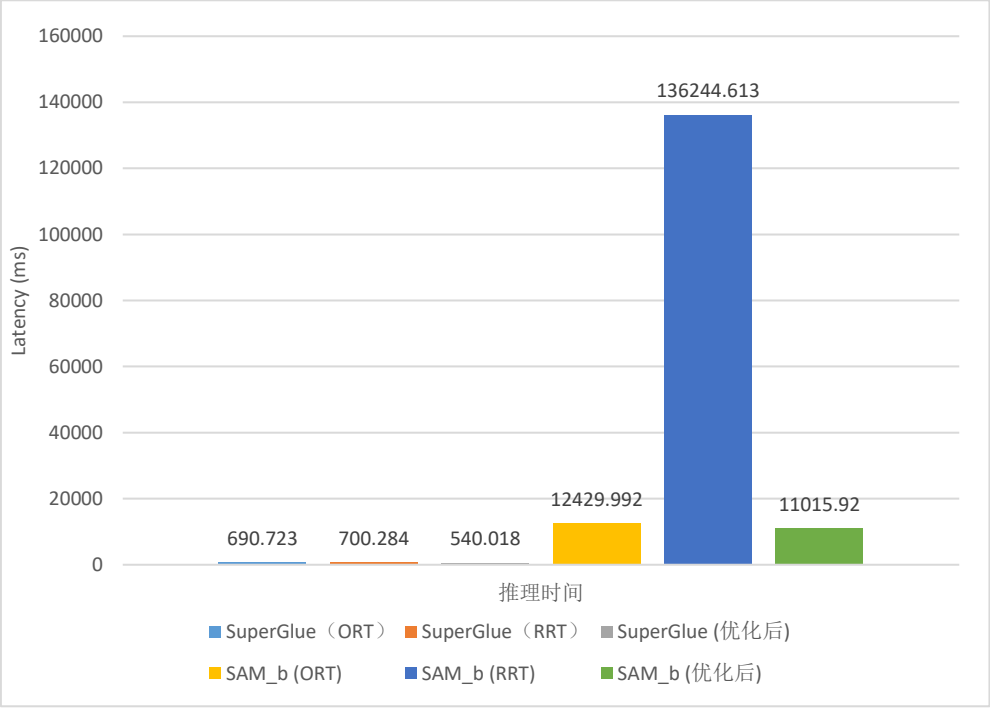


图 5-6 Transformer 优化后推理时间对比

（2）单一 CNN 模型实验结果与分析

图 5-7 为另外三种以 CNN 结构为基础的图像融合、目标检测和图像分类模型的对比结果。由图可知，对于 CNN 模型来说，优化前后推理时间没有明显变化。相对于原先的最佳推理框架的推理时间结果，相比于 RRT 推理，Yolov8n 模型使用降维与模型分区后推理时间升高 1.54%，ResNet50 模型推理时间降低 0.51%，SeAfusion 模型推理时间升高 1.32%。

这个结果说明模型分区与高维算子降维可对于 CNN 模型几乎没有作用。这是因为 CNN 模型中几乎不含有高维算子，并且包含大量卷子算子，使得几乎全部的算子使用 RRT 推理可以以少量数据传输代价获得相比于 ORT 的推理时间的大幅降低。因此模型分区几乎不会对 CNN 模型中的算子剥离出来使用 CPU 执行。

中国知网 https://www.cnki.net

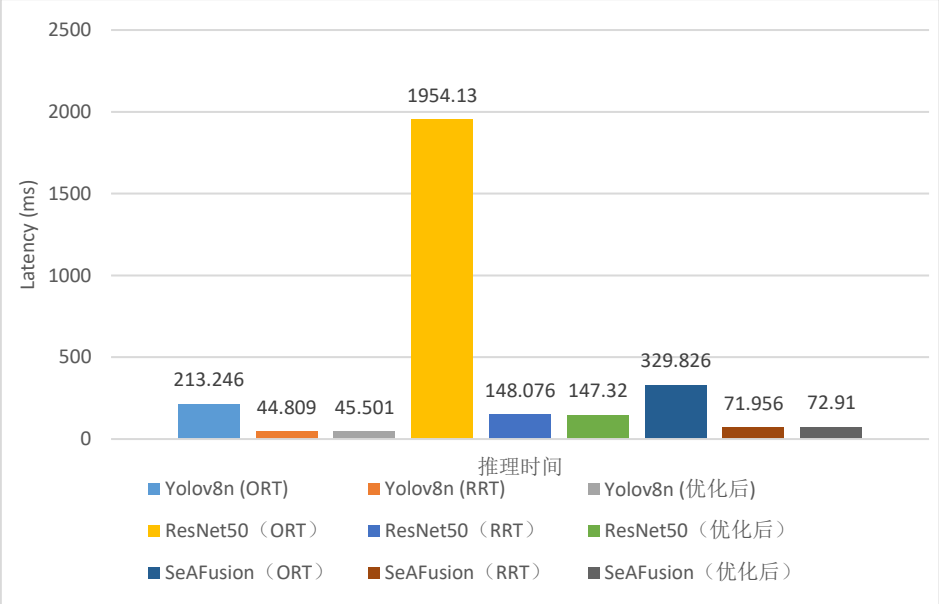


图 5-7 CNN 模型优化后推理时间对比

（3）多模型实验结果与分析

对于多模型场景，根据应用需求，需要同时运行图像分割、目标检测、图像融合等应用，为了证明本文提出的策略的有效性，采取两种调度策略作为对比，分别为：策略 1：所有模型使用各自最优推理框架并使用 Debian11 操作系统默认调度策略进行推理，策略 2：使用本文的优化方法将所有模型优化之后，结合 4.3 提出的调度策略进行混合推理。最后进行实验分析并得出结论。

为了在各自调度策略下具有最优表现，实验测试采用最短作业优先方式依次启动四种应用。但之后应用只有当执行完毕以后才能马上发起下一次请求，等待被下一次调度。此外，策略测试一次时长为 10 分钟，并对策略一与策略二分别取多次测试结果取平均，得到本文最终实验结果。

图 5-8 为 5 种模型采用策略一与策略二进行混合调度部署的平均响应时间结果对比。由图可知，相比于策略 1，策略 2 极大地降低了模型的平均响应时间，特别是 SAM_b 模型，平均响应时间缩短 85.61%，Yolov8n 平均减小 68.29%，SuperGlue 平均减小 66.23%，ResNet50 时间缩短 60.5%，而 SeAFusion 缩短 63.81%。

这是因为策略 1 使用的默认调度策略为先来先服务策略，当一个进程需要 CPU 计算资源时，它将会一直占用 CPU 直到程序执行结束。SAM_b 模型执行时会消耗大量的 CPU 资源，并且它每一帧推理时间都比其他 AI 任务长，因此一旦获得 CPU 资源，其他 AI 进程将会长时间处于阻塞状态。而策略 2 使用模型分区算法与基于 EDF 的调度相结合的策略，使得 SAM_b 模型并不总是占用 CPU 资源，在使用 NPU

中国知网 https://www.cnki.net

执行子模型时，其他 AI 进程将会获得 CPU 资源，从而极大提高了其他 AI 进程的响应效率，表现为平均响应时间减少、平均响应次数增加。

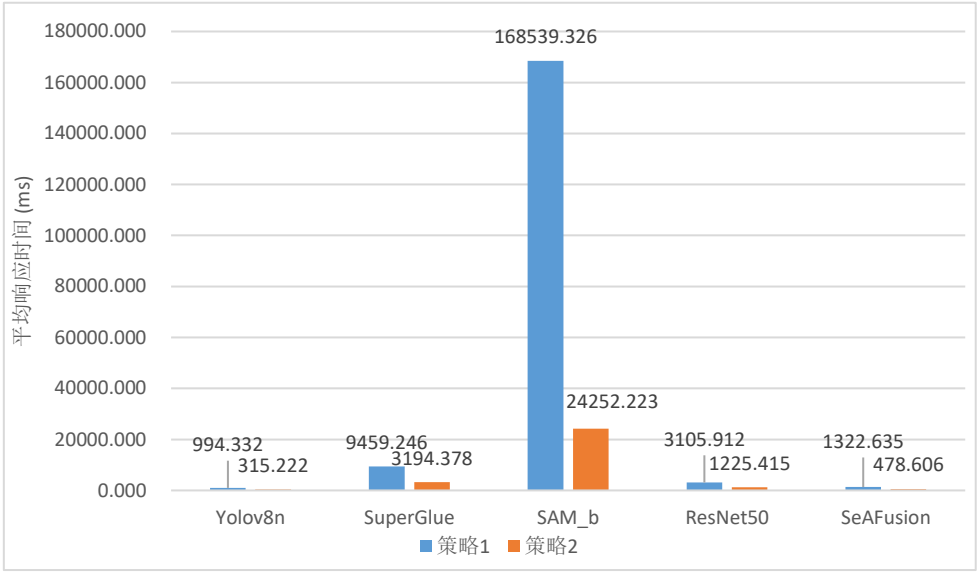


图 5-8 模型平均响应时间对比

这样的结果得益于 SAM_b 经过算子降维之后性能获得的巨大提升，以及合理的基于 EDF 的实时多任务调度策略。

表 5-2 为策略一与策略二中系统总体的实验结果对比，其中策略二系统总体平均响应时间为 777.351ms。相较于策略一，本文提出的策略二平均响应时间减少约 65.86%。策略二极大提高了 NPU 利用率，相对策略一提高了 148%，CPU 利用率降低了 14.68%，这对于移动端低功耗场景也是非常有益的。

表 5-2 算法总体性能对比

调度策略	总平均响应时间(ms)	CPU 利用率(%)	NPU 利用率(%)
策略 1	2277.506	76.376	27.22
策略 2	777.351	65.16	68.53

5.4 本章小结

本章介绍了一个专为高效模型优化与部署而设计的平台，并基于这个平台进行实验与分析。该平台结合了特定的硬件配置和定制化的软件系统，满足了不同类型深度学习模型的需求。实验针对当前主流的 Transformer 类模型与 CNN 类模型进行了深入的测试，最终实验结果表明，使用本文提出的混合任务调度策略可以减少约 65%的平均响应时间。本章的研究成果对于推动移动端深度学习应用的发展具有重要意义，同时也为未来相关领域的研究提供了有价值的参考。

中国知网 https://www.cnki.net

结　论

本文围绕RK3588异构融合处理器的特点及其现有问题，成功地构建了一个深度学习模型部署平台，该平台能够最大化利用 CPU 和 NPU 的计算资源，显著提升了深度学习模型在移动端设备上的推理速度。以下是具体的研究成果及贡献

（1）通过对 RK3588 异构融合处理器硬件平台以及两大主要模型后端推理框架（ONNX Runtime 与 RKNN Runtime）的全面而深入的分析，识别出了不同计算规模下各类算子的表现差异，并发现了现有 CPU 与 RKNPU 推理框架中存在的潜在问题。这些问题为后续研究提供了明确的方向，帮助优化了模型部署流程。

（2）针对 Transformer 模型中存在超过四维度的算子且这些高维算子难以直接部署到 RKNPU 的问题，本文提出了一种创新性的算子降维算法。通过追溯造成算子形状达到或超过五维的原因——通常是 Reshape 与 Transpose 操作，本文的方法有效地将这些高维算子进行了降维处理，从而实现了 NPU 计算资源更为合理的使用。这一技术突破不仅解决了特定类型算子难以适应 NPU 架构的问题，也为其他类似问题提供了新的解决思路。

（3）鉴于 RKNPU 与 CPU 推理框架之间存在的性能差异，尤其是在处理 Transpose 算子时表现尤为明显，再加上 RKNN 模型转换过程中可能出现自动添加 Transpose 的情况，提出了一种基于冗余算子组合的纯净矩阵乘法算子转换方法。此算法可以实现 Transpose 算子与 NPU 之间的完全分离，避免不必要的计算开销，提高了整体效率。此外，这种方法还促进了模型结构简化，减少了模型复杂度，进一步增强了模型在不同硬件平台上的移植性和兼容性。

（4）针对 RK3588 上两种深度学习模型推理框架不能充分利用计算资源计算某些特殊算子的问题，采取了一系列措施来提升计算资源利用率和模型推理时间。首先，采用零拷贝技术以减少数据传输所需的时间；其次，利用机器学习回归预测建立了一个数据传输时延预测模块，用于指导模型分区决策；最后，引入内存复用技术和基于 EDF 的多模型调度策略，确保了 RK3588 能够在有限的内存条件下支持更大规模的模型部署，并提高了任务处理的速度和响应能力。

本文后续研究工作可从以下几方面进行：

（1）通过混合量化的方式在保证模型精度不下降的前提下进一步提高模型在移动端的推理速度。

（2）通过自定义算子将 Transpose、Reshape 等算子推理用更高效的方式实现。

中国知网　https://www.cnki.net

（3）通过动态频率调节技术 DVFS 合理地调控 NPU 与 CPU 的频率与电压达到节省功耗的目的。

中国知网 https://www.cnki.net

参考文献

[1] HE KAIMING, ZHANG XIANGYU, REN SHAOQING, et al. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification[C]. //2015 IEEE International Conference on Computer Vision (ICCV), 2015: 1026-1034.

[2] ALI M, ZHANG Z. The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection[J]. Computers, 2024, 13(12): 336 .

[3] LIU X, HE P, CHEN W, et al. Multi-Task Deep Neural Networks for Natural Language Understanding [C]// A. Korhonen, D. Traum, L. Márquez. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Florence, Italy: Association for Computational Linguistics, 2015: 4487-4496.

[4] XIONG W, DROPPO J, HUANG X, et al. Toward Human Parity in Conversational Speech Recognition[J]. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 2017: 2410–2423.

[5] CHOROWSKI J K, BAHDANAU D, SERDYUK D, et al. Attention-based models for speech recognition[J]. Advances in neural information processing systems, 2015, 28.

[6] PARK J, BOO Y, CHOI I, et al. Fully neural network based speech recognition on mobile and embedded devices[C]. //Proceedings of the 32nd International Conference on Neural Information Processing Systems, 2018: 10642–10653.

[7] SATTLER T, HAVLENA M, SCHINDLER K, et al. Large-Scale Location Recognition and the Geometric Burstiness Problem[C]. //2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016: 1582-1590.

[8] TAIRA H, OKUTOMI M, SATTLER T, et al. InLoc: Indoor Visual Localization with Dense Matching and View Synthesis[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021: 1293–1307.

[9] ZHANG H, SONG C, WANG A, et al. PDVocal: Towards Privacy-preserving Parkinson's Disease Detection using Non-speech Body Sounds[C]. //The 25th Annual International Conference on Mobile Computing and Networking, 2019.

[10] WANG Y, YI B, YU K. Exploration and research about key technologies concerning Deep Learning models targeting Mobile Terminals[J]. Journal of Physics: Conference Series, 2022, 2303(1): 012086.

[11] HU J, SHEN L, ALBANIE S, et al. Squeeze-and-Excitation Networks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020: 2011–2023.

中国知网 https://www.cnki.net

[12] LI Y, YUAN G, WEN Y, et al. EfficientFormer: vision transformers at MobileNet speed[C]. //Proceedings of the 36th International Conference on Neural Information Processing Systems, 2024.

[13] HAN K, WANG Y, TIAN Q, et al. GhostNet: More Features From Cheap Operations[C]. //2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020: 1577-1586.

[14] LI Y, CHEN Y, DAI X, et al. MicroNet: Improving Image Recognition with Extremely Low FLOPs[C]. //2021 IEEE/CVF International Conference on Computer Vision (ICCV), 2021: 458-467.

[15] HAN S, MAO H, DALLY WILLIAMJ. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding[C]//ICLR, 2015.

[16] TANAKA H, KUNIN D, YAMINS D L, et al. Pruning neural networks without any data by iteratively conserving synaptic flow[J]. Advances in neural information processing systems, 2020, 33: 6377-6389.

[17] WEN W, WU C, WANG Y, et al. Learning structured sparsity in deep neural networks[C]. //Proceedings of the 30th International Conference on Neural Information Processing Systems,2016: 2082–2090.

[18] JACOB B, KLIGYS S, CHEN B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[J]. 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2017: 2704-2713.

[19] HUBARA I, COURBARIAUX M, SOUDRY D, et al. Quantized neural networks: training neural networks with low precision weights and activations[J]. J. Mach. Learn. Res. 2017, 18(1): 6869–6898.

[20] ADRINANAR, NICOLAS B, EBRAHIMI K S, et al. Fitnets: Hints for thin deep nets[J]. Proc. ICLR, 2015, 2(3): 1.

[21] ZHANG C, HAN D, QIAO Y, et al. Faster Segment Anything: Towards Lightweight SAM for Mobile Applications[J]. 2023.

[22] ZHOU C, LI X, LOY C, et al. EdgeSAM: Prompt-In-the-Loop Distillation for On-Device Deployment of SAM[J]. 2023.

[23] BHATTACHARYA S, LANE N D. Sparsification and Separation of Deep Learning Layers for Constrained Resource Inference on Wearables[C]. //Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM, 2016: 176–189.

[24] FANG B, ZENG X, ZHANG M, et al. NestDNN: Resource-Aware Multi-Tenant On-Device Deep Learning for Continuous Mobile Vision[C]. //Proceedings of the 24th Annual International Conference on Mobile Computing and Networking, 2018:

中国知网 https://www.cnki.net

115–127.
[25] HAN S, POOL J, TRAN J, et al. Learning both weights and connections for efficient neural networks[C]. //Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1, 2015: 1135–1143.
[26] LEE S, NIRJON S. Deep Functional Network (DFN)[A]. Proceedings of the 20th International Conference on Information Processing in Sensor Networks (co-located with CPS-IoT Week 2021)[C]. 2021: 191–206.
[27] HAN R, ZHANG Q, LIU C, et al. LegoDNN: block-grained scaling of deep neural networks for mobile vision[C]. //Proceedings of the 27th Annual International Conference on Mobile Computing and Networking, 2021: 406–419.
[28] CHEN T, MOREAU T, JIANG Z, et al. TVM: an automated end-to-end optimizing compiler for deep learning[C]. //Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation, 2018: 579–594.
[29] ZHAO Y, SUN Q, HE Z, et al. AutoGraph: optimizing DNN computation graph for parallel GPU kernel execution[C]. //Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence, 2023.
[30] JANG J-W, LEE S, KIM D, et al. Sparsity-Aware and Re-configurable NPU Architecture for Samsung Flagship Mobile SoC[A]. 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)[C]. 2021: 15–28.
[31] PARK H, KIM S. Overviewing AI-Dedicated Hardware for On-Device AI in Smartphones[A]. 见: Artificial Intelligence and Hardware Accelerators[M]. 2023: 127–150.
[32] JUNG H, JI H, PUSHCHIN A, et al. Accelerating Deep Neural Networks on Mobile Multicore NPUs[A]. Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization[C]. 2023: 236–248.
[33] YU Z, LIANG S, MA T, et al. Cambricon-LLM: A Chiplet-Based Hybrid Architecture for On-Device Inference of 70B LLM[C]. //2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2024: 1474-1488.
[34] KIM Y, KIM J, Chae D, et al. µLayer: Low Latency On-Device Inference Using Cooperative Single-Layer Acceleration and Processor-Friendly Quantization[A]. Proceedings of the Fourteenth EuroSys Conference 2019[C]. 2019.
[35] JIA F, ZHANG D, CAO T, et al. CoDL: efficient CPU-GPU co-execution for deep learning inference on mobile devices[C]. //Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services, 2022: 209–221.

中国知网 https://www.cnki.net

[36] WANG M, DING S, CAO T, et al. AsyMo: scalable and efficient deep-learning inference on asymmetric mobile CPUs[C]. //Proceedings of the 27th Annual International Conference on Mobile Computing and Networking, 2021: 215–228.

[37] ZHANG L L, HAN S, WEI J, et al. nn-Meter: towards accurate latency prediction of deep-learning model inference on diverse edge devices[C]. //Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services, 2021: 81–93.

[38] KANG W, LEE K, LEE J, et al. LaLaRAND: Flexible Layer-by-Layer CPU/GPU Scheduling for Real-Time DNN Tasks[C]. //2021 IEEE Real-Time Systems Symposium (RTSS), 2021: 329-341.

[39] TAN T, CAO G. Efficient Execution of Deep Neural Networks on Mobile Devices with NPU[C]. //Proceedings of the 20th International Conference on Information Processing in Sensor Networks (Co-Located with CPS-IoT Week 2021), 2021: 283–298.

[40] LING N, HUANG X, ZHAO Z, et al. BlastNet: Exploiting Duo-Blocks for Cross-Processor Real-Time DNN Inference[C]. //Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems, 2023: 91–105.

[41] XIANG Y, KIM H. Pipelined Data-Parallel CPU/GPU Scheduling for Multi-DNN Real-Time Inference[C]. //2019 IEEE Real-Time Systems Symposium (RTSS),2019,:392-405.

[42] HWANG S, CHOI J, YOO S, et al. Hardware Interrupt and CPU Contention aware CPU/GPU Co-Scheduling on Multi-Cluster System[C]. //2022 5th International Conference on Information and Computer Technologies (ICICT), 2022: 117-120.

[43] LIN C, CHEN Z, ZHANG Z, et al. TOP: Task-Based Operator Parallelism for Asynchronous Deep Learning Inference on GPU[J]. IEEE Transactions on Parallel and Distributed Systems, 2025, 36(2): 266-281.

[44] LIN C, LIU J. COS: Cross-Processor Operator Scheduling for Multi-Tenant Deep Learning Inference[C]. //2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), 2024: 1-10.

[45] WANG K CAO J, ZHOU Z, et al. SwapNet: Efficient Swapping for DNN Inference on Edge AI Devices Beyond the Memory Budget[J]. IEEE Transactions on Mobile Computing, 2024, 23(9): 8935-8950.

[46] BAI, JUNJIE, LU, et al. ONNX: Open Neural Network Exchange[CP/OL]. (2019) [2025-01-05]. https://github.com/onnx/onnx

[47] ONNX RUNTIME DEVELOPERS. ONNX Runtime[CP/OL]. (2021) [2025-01-05]. https://onnxruntime.ai/

[48] ALEXANDER K, ERIC M, NIKHILA R, et al. Segment Anything[J]. 2023

中国知网 https://www.cnki.net

IEEE/CVF International Conference on Computer Vision (ICCV), 2023: 3992-4003.

[49] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]. //Proceedings of the 31st International Conference on Neural Information Processing Systems, 2017: 6000–6010.

中国知网 https://www.cnki.net